NATIONAL UNIVERSITY OF HO CHI MINH CITY
**UNIVERSITY OF SCIENCE**
**FACULTY OF ELECTRONICS AND TELECOMMUNICATIONS**
**DEPARTMENT OF ELECTRONICS**

**LÊ MINH ĐỨC**

# ALGORITHM-HARDWARE CO-DESIGN OF A HARDWARE-EFFICIENT QUANTIZED 1D CNN ACCELERATOR FOR ECG ARRHYTHMIA CLASSIFICATION ON FPGA

BACHELOR'S THESIS
MAJOR: ELECTRONICS AND TELECOMMUNICATIONS ENGINEERING
SPECIALIZATION: ELECTRONICS

**Ho Chi Minh City - 2026**

NATIONAL UNIVERSITY OF HO CHI MINH CITY
**UNIVERSITY OF SCIENCE**
**FACULTY OF ELECTRONICS AND TELECOMMUNICATIONS**

**BACHELOR'S THESIS**
**MAJOR: ELECTRONICS AND TELECOMMUNICATIONS ENGINEERING**
**SPECIALIZATION: ELECTRONICS**

# ALGORITHM-HARDWARE CO-DESIGN OF A HARDWARE-EFFICIENT QUANTIZED 1D CNN ACCELERATOR FOR ECG ARRHYTHMIA CLASSIFICATION ON FPGA

**Student name: LE MINH DUC**
**Student ID: 22200034**

SCIENTIFIC SUPERVISOR:
ThS. Mã Khải Minh

**Ho Chi Minh City - 2026**

# XÁC NHẬN HOÀN TẤT CHỈNH SỬA BÁO CÁO TỐT NGHIỆP

**TÊN ĐỀ TÀI:**

**Đồng thiết kế thuật toán–phần cứng cho bộ tăng tốc CNN một chiều lượng tử hóa hiệu quả nhằm phân loại rối loạn nhịp tim từ tín hiệu ECG trên FPGA**

Tên tiếng Anh:

Algorithm-hardware co-design of a hardware-efficient quantized 1D CNN accelerator for ecg arrhythmia classification on FPGA

Họ và tên sinh viên: Lê Minh Đức

Mã số sinh viên: 22200034

**Hội đồng đánh giá họp ngày 15 tháng 8 năm 2026**

1. Chủ tịch:
2. Thư ký:
3. Ủy viên:
4. Ủy viên:
5. Ủy viên:

Người hướng dẫn:

Xác nhận của người hướng dẫn:

# LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Mã Khải Minh, người hướng dẫn khoa học của em. Thầy đã giúp em định hình hướng đi của đề tài, cho em những góp ý chuyên môn quý báu và đồng hành cùng em trong suốt quá trình học tập cũng như thực hiện nghiên cứu và hoàn thành khoá luận.

Em xin bày tỏ lòng biết ơn đến quý thầy cô Bộ môn Điện tử, Khoa Điện tử – Viễn thông, Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh, những người đã tận tâm giảng dạy, truyền đạt kiến thức và khơi dậy niềm say mê học tập cho sinh viên. Sự tận tuỵ của thầy cô chính là nền tảng vững chắc giúp em hoàn thành khoá luận này.

Em xin cảm ơn gia đình đã luôn tạo mọi cơ hội và điều kiện để em tiếp tục con đường học tập. Con cảm ơn ba mẹ đã luôn dõi theo, ủng hộ, tin tưởng và động viên con trên hành trình theo đuổi ước mơ của mình.

Cuối cùng, em không thể không nhắc đến những người bạn đã luôn ở bên cạnh em, không chỉ là chỗ dựa tinh thần mà còn cùng em chia sẻ những kiến thức mà có lẽ em đã không có được nếu chỉ ngồi trên giảng đường. Nhờ sự đồng hành, động viên và giúp đỡ của mọi người, em đã có thêm động lực để làm việc, học tập và hoàn thành khoá luận.

# LỜI CAM KẾT

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

Sinh viên thực hiện

Lê Minh Đức

# TÓM TẮT

Việc theo dõi điện tâm đồ (ECG) liên tục trên thiết bị đeo đòi hỏi các mô hình học máy vừa đạt độ chính xác phân loại cao, vừa hiện thực được hiệu quả trên phần cứng. Khoá luận này trình bày hướng tiếp cận đồng thiết kế thuật toán – phần cứng cho bài toán phân loại rối loạn nhịp tim từ tín hiệu ECG trên FPGA, sử dụng một bộ tăng tốc mạng nơ-ron tích chập một chiều (1D-CNN) đã lượng tử hoá theo hướng thân thiện với phần cứng. Khung thiết kế đề xuất tối ưu đồng thời mạng nơ-ron và kiến trúc phần cứng thông qua cắt tỉa kênh có cấu trúc và lượng tử hoá INT8 với thang luỹ thừa của hai, nhờ đó phép tái lượng tử hoá chỉ cần đến các phép dịch bit. Một bộ tăng tốc RTL tự thiết kế gồm tám khối convolution-pool unit chạy song song, khối gap-fc-argmax, bộ điều khiển và kiến trúc bộ nhớ ping-pong trên chip được hiện thực và kiểm chứng bằng cách so sánh khớp bit với mô hình tham chiếu phần mềm trước khi triển khai lên FPGA. Hệ thống đề xuất đạt độ chính xác 94,27% với F1-macro 0,9356 trên tập Chapman và 93,00% khi đánh giá chéo trên tập Georgia. Khi hiện thực trên FPGA DE10-Standard, bộ tăng tốc chiếm 2.148 ALM và 28 khối DSP, hoạt động ở tần số tối đa 108,86 MHz, thực hiện một lần suy luận trong 52,16 µs ở 100 MHz với mức tiêu thụ năng lượng ước lượng 27,96 µJ. Các kết quả này cho thấy đồng thiết kế thuật toán – phần cứng cho phép triển khai hiệu quả các mạng nơ-ron sâu gọn nhẹ lên nền tảng FPGA hạn chế tài nguyên, đạt đồng thời độ chính xác cao, độ trễ thấp và tiết kiệm năng lượng, phục vụ theo dõi ECG thời gian thực trên thiết bị đeo.



**Từ khoá:** Đồng thiết kế thuật toán – phần cứng, Hiệu quả phần cứng, Lượng tử hoá, Bộ tăng tốc.

## ABSTRACT

Continuous electrocardiogram (ECG) monitoring on wearable devices requires machine learning models that simultaneously achieve high classification accuracy and efficient hardware implementation. This thesis presents an algorithm–hardware co-design approach for FPGA-based ECG arrhythmia classification using a hardware-efficient quantized one-dimensional convolutional neural network (1D-CNN) accelerator. The proposed framework jointly optimizes the neural network and hardware architecture through structured channel pruning and INT8 quantization with power-of-two scaling, enabling requantization using only bit-shift operations. A custom RTL accelerator comprising eight parallel convolution-processing units, a GAP-FC-Argmax module, a controller, and an on-chip ping-pong memory architecture is implemented and verified using bit-exact comparison with the software reference model before FPGA deployment. The proposed system achieves 94.27% accuracy with a macro-F1 score of 0.9356 on the Chapman dataset and 93.00% accuracy under cross-dataset evaluation on the Georgia dataset. Implemented on a DE10-Standard FPGA, the accelerator occupies 2,148 ALMs and 28 DSP blocks, operates at up to 108.86 MHz, and performs one inference in 52.16 μs at 100 MHz with an estimated energy consumption of 27.96 μJ. These results demonstrate that algorithm–hardware co-design effectively enables accurate, low-latency, and energy-efficient deployment of compact deep neural networks on resource-constrained FPGA platforms for real-time wearable ECG monitoring.



**Keywords:** Algorithm-Hardware Co-Design, Hardware-Efficient, Quantization, Accelerator.

# TABLE OF CONTENTS

TÓM TẮT ..... i
TABLE OF CONTENTS ..... iii
LIST OF ABBREVIATIONS ..... v
LIST OF FIGURES ..... vi
LIST OF TABLES ..... vii
CHAPTER 1: INTRODUCTION ..... 1
1.1. BACKGROUND AND OBJECTIVES ..... 1
1.2. SCOPE AND SUBJECT OF THE STUDY ..... 2
1.3. RESEARCH METHODOLOGY ..... 3
1.4. TOOLS AND TARGET BOARD ..... 4
1.5. THESIS STRUCTURE ..... 5
CHAPTER 2: THEORETICAL BACKGROUND ..... 6
2.1. ECG SIGNALS AND CARDIAC ARRHYTHMIAS ..... 6
2.1.1. The electrocardiogram signal ..... 6
2.1.2. The Chapman training set and the Georgia cross-dataset test set ..... 6
2.2. OVERVIEW OF CONVOLUTIONAL NEURAL NETWORKS ..... 8
2.2.1. Neural, convolutional and one-dimensional networks ..... 8
2.2.2. The 1D-CNN model layer by layer ..... 9
2.3. MODEL TRAINING ..... 10
2.4. QUANTIZATION ..... 12
2.5. HARDWARE ARCHITECTURE ..... 14
CHAPTER 3: DESIGN AND IMPLEMENTATION OF THE CNN ..... 15
3.1. DATA PROCESSING AND MODEL TRAINING WORKFLOW ..... 16
3.1.1. Data processing ..... 16
3.1.2. Model training ..... 17
3.1.3. Weight quantization ..... 19
3.1.4. Weight extraction ..... 21
3.2. DESIGN OF THE CNN SYSTEM ..... 22
3.2.1. Design of the convolution-pool unit ..... 23
3.2.2. Design of the engine unit ..... 25
3.2.3. Design of the gap-fc-argmax unit ..... 29
3.2.4. Design of the controller and the auxiliary blocks ..... 30
3.2.5. Dataflow of the CNN system ..... 34
3.3. INTERFACE AND SYSTEM CONTROL INTEGRATION ..... 35

3.3.1. Integration of the Avalon-Memory Mapped Wrapper with the CNN core ..... 35
3.3.2. Integration system with the JTAG-to-Avalon IP to communicate with a PC ..... 35
CHAPTER 4: IMPLEMENTATION RESULTS AND EVALUATION ..... 37
4.1. EVALUATION METHODOLOGY ..... 37
4.1.1. Model performance ..... 37
4.1.2. Hardware performance ..... 38
4.2. MODEL TRAINING RESULTS ..... 39
4.2.1. Results on the training dataset ..... 39
4.2.2. Results on the cross-dataset test set ..... 41
4.3. FUNCTIONAL SIMULATION RESULTS ..... 42
4.3.1. Functional simulation of the convolution–pool unit ..... 42
4.3.2. Functional simulation of layer-by-layer operation ..... 43
4.3.3. Simulation of test-set samples ..... 44
4.3.4. Simulation of cross-dataset samples ..... 45
4.4. EXPERIMENTAL RESULTS ON THE FPGA BOARD ..... 45
4.4.1. Results for a single sample ..... 46
4.4.2. Results for the complete test set ..... 46
4.5. RESOURCE AND PERFORMANCE EVALUATION ..... 46
4.5.1. Resource results ..... 46
4.5.2. Performance results ..... 48
4.6. COMPARISON WITH RELATED WORK ..... 49
CHAPTER 5: CONCLUSION ..... 51
5.1. ACHIEVED RESULTS ..... 51
5.2. LIMITATIONS ..... 52
5.3. FUTURE WORK ..... 53
REFERENCES ..... 54

## LIST OF ABBREVIATIONS

| | |
|---|---|
| **AFIB** | Atrial Fibrillation |
| **ALM** | Adaptive Logic Module |
| **AUC** | Area Under the Curve |
| **CNN** | Convolutional Neural Network |
| **DSP** | Digital Signal Processing |
| **ECG** | Electrocardiogram |
| **FC** | Fully Connected |
| **FPGA** | Field-Programmable Gate Array |
| **GAP** | Global Average Pooling |
| **GSVT** | General Supraventricular Tachycardia |
| **HDL** | Hardware Description Language |
| **INT8** | Integer 8-bit |
| **IP** | Intellectual Property |
| **JTAG** | Joint Test Action Group |
| **LSB** | Least Significant Bit |
| **LUT** | Look-Up Table |
| **PTQ** | Post-Training Quantization |
| **QAT** | Quantization-Aware Training |
| **QRS** | QRS complex |
| **RAM** | Random Access Memory |
| **ReLU** | Rectified Linear Unit |
| **ROC** | Receiver Operating Characteristic |
| **ROM** | Read-Only Memory |
| **RTL** | Register Transfer Level |
| **SB** | Sinus Bradycardia |
| **SoC** | System on Chip |
| **SR** | Sinus Rhythm |

## LIST OF FIGURES

Figure 1.1. Main components of the DE10-Standard kit. ..... 5
Figure 2.1. Typical ECG waveforms of the four rhythm classes; each trace is one 10-second record from the test set. ..... 7
Figure 3.1. Design flow from software model to FPGA demonstration. ..... 16
Figure 3.2. Software flow from raw data to the hardware weight set. ..... 16
Figure 3.3. CNN architecture before pruning (1,244 parameters). ..... 18
Figure 3.4. CNN after pruning (640 parameters) — the hardware configuration. ..... 19
Figure 3.5. Architecture of the CNN system of this work. ..... 23
Figure 3.6. The convolution-pool unit and its three sub-blocks. ..... 24
Figure 3.7. Engine unit flow diagram. ..... 26
Figure 3.8. The gap-fc-argmax unit. ..... 30
Figure 3.9. State diagram of the controller. ..... 31
Figure 3.10. Dataflow through the four convolutional layers and the bank swap. ..... 35
Figure 3.11. On-board integration with the JTAG-to-Avalon bridge. ..... 37
Figure 4.1. Loss (a) and accuracy (b) curves for the training and validation sets ..... 40
Figure 4.2. Confusion matrix (a) and ROC curves (b) on the Chapman test set. ..... 41
Figure 4.3. Confusion matrix (a) and ROC curves (b) on the Georgia set. ..... 42
Figure 4.4. Waveforms of the convolution-pool unit. ..... 44
Figure 4.5. Waveforms of the transition between convolutional layers. ..... 45
Figure 4.6. Waveforms of one complete bit-accurate inference. ..... 46
Figure 4.7. Waveforms of a Georgia cross-dataset sample. ..... 46
Figure 4.8. The full test set on the board, shown in the System Console. ..... 47
Figure 4.9. Resources of the hardware design and of the board bitstream ..... 48

## LIST OF TABLES

Table 2.1. Mapping of original labels onto the four classes ..... 7
Table 2.2. Records per class in the Chapman dataset ..... 7
Table 3.1. Dataset partitioning. ..... 17
Table 3.2. Channels per layer before and after structured pruning. ..... 19
Table 3.3. Layer configuration and data-size transformations of the pruned model. ..... 19
Table 3.4. Power-of-two quantization parameters for each layer. ..... 21
Table 3.5. Weight files loaded into the hardware ROMs. ..... 22
Table 3.6. List of RTL modules in the design. ..... 23
Table 3.7. Port interface of the convolution-pool unit. ..... 24
Table 3.8. Pipeline stages of the convolution-pool unit. ..... 25
Table 3.9. Port interface of the engine unit. ..... 27
Table 3.10. Mapping of sliding-window cells to multiplication branches. ..... 27
Table 3.11. Conditions generating zero padding. ..... 28
Table 3.12. Five-cycle pipeline depth, selector to accumulator register. ..... 28
Table 3.13. Cycle budget of the gap-fc-argmax unit. ..... 30
Table 3.14. Port interface of the gap-fc-argmax unit. ..... 30
Table 3.15. States of the controller. ..... 31
Table 3.16. Control parameters for each layer. ..... 32
Table 3.17. Five sub-states of the output classification stage. ..... 33
Table 3.18. On-chip memory organization. ..... 34
Table 3.19. Cycle-count analysis of a single inference. ..... 35
Table 3.20. Avalon-MM address map of the hardware design. ..... 36
Table 3.21. System control steps performed by a script in the System Console. ..... 37
Table 4.1. Accuracy at each processing step (Chapman test set, 4,973 records) ..... 40
Table 4.2. Precision, recall and F1-score for each class (Chapman) ..... 41
Table 4.3. Precision, recall and F1-score for each class (Georgia) ..... 42
Table 4.4. float32 vs INT8 on the Georgia cross-dataset set ..... 42
Table 4.5. Test plan for the convolution-pool unit ..... 43

Table 4.6. Test plan for layer operation....44
Table 4.7. Bit-accuracy test plan using test-set samples ....45
Table 4.8. Single-sample classification on the board ....47
Table 4.9. On-board vs software accuracy ....47
Table 4.10. Resources per design block ....48
Table 4.11. Resource use relative to the Cyclone V 5CSXFC6D6F31C6 capacity....48
Table 4.12. Operating frequency after synthesis ....49
Table 4.13. Cycle-count analysis for each layer....49
Table 4.14. Latency, throughput and energy per inference ....50
Table 4.15. Comparison with related work on ECG-based arrhythmia classification ..51

# CHAPTER 1: INTRODUCTION

## 1.1. BACKGROUND AND OBJECTIVES

Cardiac arrhythmia is one of the leading causes of sudden cardiac death. Many arrhythmias — atrial fibrillation being the most representative — are paroxysmal in nature: they occur intermittently and may therefore go undetected during a single electrocardiographic (ECG) examination at a medical facility. Conventional ECG analysis, which relies on visual interpretation by a physician, is both demanding in terms of specialist resources and unsuitable for continuous long-term monitoring. What is required is an automated cardiac monitoring system that operates directly on a wearable or edge-computing device.

Convolutional neural networks (CNNs) classify arrhythmias from ECG with an accuracy approaching that of cardiologists [4], but most high-performance models have many parameters: microcontrollers lack real-time throughput, while GPUs exceed the power and size limits of wearables. The obstacle is thus not a still more accurate model, but fitting an already accurate model into an acceptable device form. This thesis designs a circuit dedicated to the exact network to be run: when layer count, filter sizes and channel counts are known in advance, the circuit follows the network structure closely, keeps all weights on chip, and omits the control logic needed only for generality. An FPGA suits such a circuit because it allows parallelism at the arithmetic level, gives deterministic latency, and lets resources, frequency and power be measured directly on real hardware.

This thesis aims to design and implement on an FPGA a dedicated hardware core for a 1D-CNN classifying arrhythmias from ECG signals, covering the full path from software model to a circuit running on the board, in four groups of tasks:

**Developing a 1D-CNN model compact enough for on-chip deployment:**

– Training a model that classifies four rhythm groups from a single-lead ECG with accuracy comparable to related work.

– Pruning channels so that all weights fit in on-chip memory, with no external memory.

**Quantizing the model in a hardware-friendly manner:**

– Converting the model to 8-bit integers with power-of-two scaling, so rescaling becomes a bit shift needing no multiplier.

– Building a reference model that reproduces the circuit's exact integer sequence, as the golden reference.

**Designing and verifying the hardware architecture:**

– Describing the architecture in Verilog block by block: convolution with max-pooling, controller, on-chip memories and output classifier.

– Showing the circuit is bit-accurate against the reference at many checkpoints.

**Deploying and evaluating the design on real hardware:**

– Synthesizing for a Cyclone V FPGA, programming the DE10-Standard kit and running inference on real data; reporting resources, maximum frequency, latency, power and energy per inference.

## 1.2. SCOPE AND SUBJECT OF THE STUDY

The subject comprises two coupled parts. The first is a one-dimensional convolutional network classifying four arrhythmia groups from a single-lead ECG, in its pruned and 8-bit integer form so that it runs on chip. The second is the hardware core implementing exactly that model on an FPGA: the architecture of the computational blocks, the on-chip memory organization and the method of verifying the circuit against the reference model.

The problem is posed as whole-record classification: each fixed-length window is assigned to one of the four rhythm groups rather than classifying beats after R-peak detection. This removes the noise-sensitive beat-segmentation stage, which would need separate hardware, and lets the circuit work on a continuous sample stream, as in wearable monitoring.

The model is trained on Chapman and cross-validated on Georgia to assess generalization to data from another source; both are public datasets, and no patient signals

were collected here. The input uses one lead only, consistent with wearables that cannot carry a full electrode system.

The thesis presents a single hardware design: weights preloaded in on-chip ROM and the network configuration fixed at synthesis. Verification aims to show bit-accurate agreement with the reference model and to measure the physical metrics on the board.

## 1.3. RESEARCH METHODOLOGY

The work proceeds in sequence from the software model down to the hardware circuit, with the result of each step forming a binding constraint on the next:

- **Data preparation: reading the ECG dataset, mapping labels onto four classes, extracting one lead and normalizing amplitude.**
- Model training: constructing the 1D-CNN and training it with a cross-entropy loss, the result of which serves as the reference accuracy baseline.
- Model compaction: pruning channels by weight magnitude, then fine-tuning to recover lost accuracy.
- Quantization: converting the model to 8-bit integers with power-of-two scaling, using quantization-aware training and round-half-up rounding.
- **Golden-data generation: exporting the quantized weights to files for loading into the on-chip memories, and generating reference values at checkpoints inside the network.**
- Hardware design: describing the architecture in Verilog as functional blocks and verifying each block individually before integration.
- Functional verification: simulating the complete system and comparing the values at every checkpoint with the reference model, requiring exact bit-level agreement.
- Synthesis and analysis: synthesizing for a Cyclone V FPGA, timing analysis for the maximum frequency, and power estimation.
- Board deployment: programming the DE10-Standard kit, sending ECG data from a host and comparing the results with the software model.

This methodology yields three main contributions. First, on quantization, power-of-two scaling has been used before but usually with an arithmetic right shift, which always

rounds one way; this thesis uses round-half-up, keeping the same hardware cost while making the error symmetric about zero so it does not accumulate across layers. Second, on verification, the work goes beyond comparing statistical accuracy — which can hide mutually compensating deviations — and compares values at checkpoints throughout the network, requiring exact bit-level agreement. Third, on deployment, every performance figure reported is measured on the synthesized design programmed onto the board.

The study thus draws a broader conclusion: for biomedical signal classification with compact models, co-designing model and hardware from the outset keeps all computation on chip and gives latency and energy suited to wearables, with accuracy comparable to the original floating-point model.

## 1.4. TOOLS AND TARGET BOARD

The software was written in Python with PyTorch for training, pruning and quantization. The hardware was described in Verilog, verified in ModelSim/Questa, and synthesized and timing-analyzed with Intel Quartus Prime; power was estimated with the Power Analyzer from a full-system simulation activity file.

The board is the Terasic DE10-Standard kit, built around the Intel Cyclone V SoC 5CSXFC6D6F31C6 [22] — an SoC FPGA family integrating a hard processor system beside the programmable logic. Figure 1.1 shows the main components in three colours: FPGA fabric, hard processor system and shared parts. The main resources are:

– Device: Intel Cyclone V SoC 5CSXFC6D6F31C6, speed grade C6, with programmable logic and a dual-core ARM Cortex-A9 processor system.
– Programmable logic: 41,910 ALMs
– On-chip memory: 553 M10K blocks, about 5,570 Kbit in total.
– Dedicated multipliers: 112 DSP blocks with signed multiply-accumulate.
– Peripherals: USB-Blaster II, Gigabit Ethernet, USB, UART-to-USB, microSD, VGA and an analogue-to-digital converter.

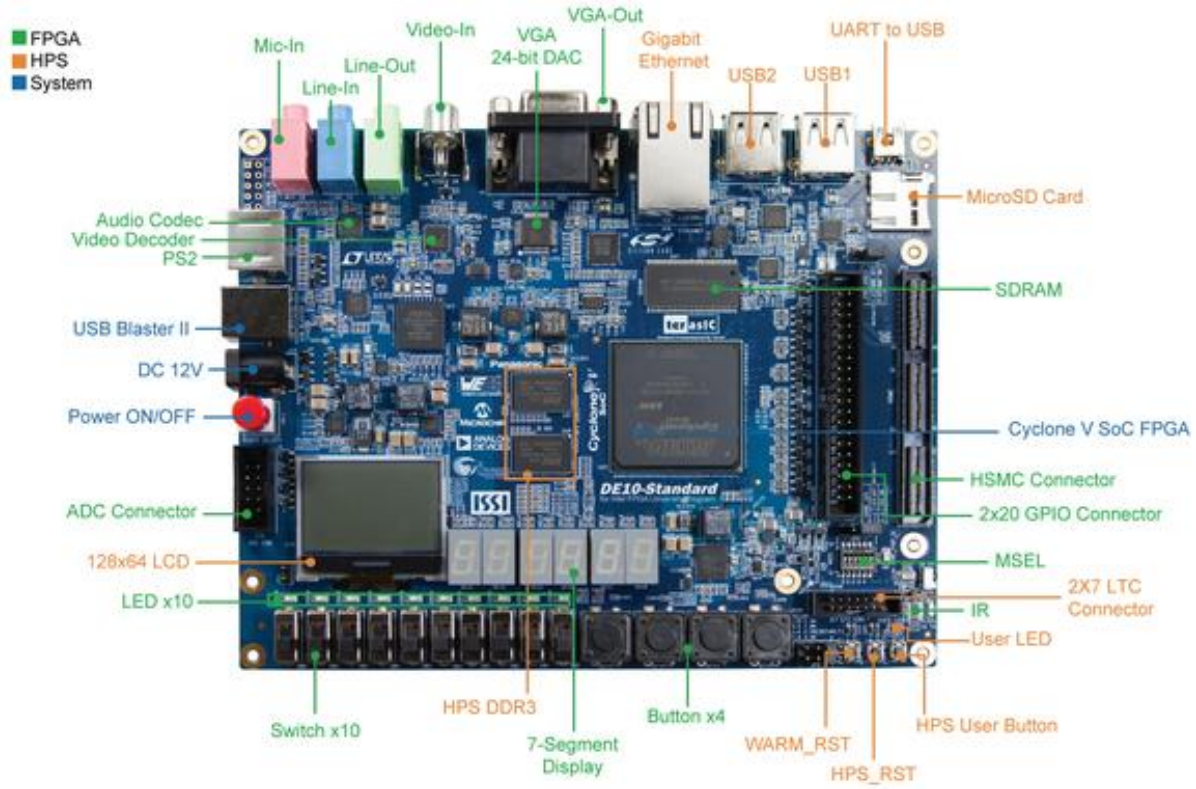



Figure 1.1. Main components of the DE10-Standard kit.

## 1.5. THESIS STRUCTURE

The thesis has five chapters:

**Chapter 1 — Introduction.** Background, objectives, scope, methodology and tools.

**Chapter 2 — Theoretical background.** The ECG signal and arrhythmia groups, one-dimensional convolutional networks, pruning and integer quantization, FPGA architecture.

**Chapter 3 — Design and implementation of the CNN**. Data processing and model training, and the hardware architecture block by block.

**Chapter 4 — Implementation results and evaluation.** Training results, cross-dataset validation, synthesis, on-board execution and comparison with related work**.**

**Chapter 5 — Conclusion.** Summary of results, limitations and directions for future work.

# CHAPTER 2: THEORETICAL BACKGROUND

## 2.1. ECG SIGNALS AND CARDIAC ARRHYTHMIAS

### 2.1.1. The electrocardiogram signal

The electrocardiogram (ECG) records the potential difference produced by the electrical activity of the heart muscle, acquired with skin electrodes. Each contraction cycle gives a repeating waveform: the P wave (atrial depolarization), the QRS complex (ventricular depolarization) and the T wave (ventricular repolarization). The standard clinical record has 12 leads; lead II serves for rhythm analysis because its axis nearly matches impulse propagation from the sinus node to the apex, giving the clearest P wave and QRS complex.

The ECG carries two complementary feature groups. Morphological features are the shapes of the waves within a cycle — a missing P wave signals atrial fibrillation, a widened QRS complex suggests an intraventricular conduction disturbance. Rhythmic features are the R–R intervals between cycles, reflecting heart rate and regularity. An arrhythmia is a rhythm that loses its regularity or leaves the physiological frequency range: sinus tachycardia and bradycardia differ from normal sinus rhythm mainly by frequency thresholds, while atrial fibrillation shows irregularly varying R–R intervals.

### 2.1.2. The Chapman training set and the Georgia cross-dataset test set

Two independent datasets serve two purposes: one for training and in-distribution evaluation, one for cross-dataset testing of generalization to data from a different system.

The training set is Chapman [1], consolidated from two related collections on PhysioNet. Each record is a 12-lead ECG of 10 s sampled at 500 Hz with a physician-assigned rhythm label. After removing records duplicated between the two sources to avoid leakage between subsets, 33,143 records remain, split into training, validation and test subsets.

The original labels are detailed diagnostic codes, grouped into four classes following the taxonomy of the dataset authors. Table 2.1 gives the grouping rules, keeping the original English label names; Figure 2.1 shows representative waveforms of the four classes, with their differences in heart rate and R–R regularity.

Table 2.1. Mapping of original labels onto the four classes

| Class | Original label (English name) | Description |
|---|---|---|
| AFIB | Atrial Fibrillation (AFIB), Atrial Flutter (AF) | Atrial fibrillation and atrial flutter |
| GSVT | Sinus Tachycardia (ST), Supraventricular Tachycardia (SVT), Atrial Tachycardia (AT), AVNRT, AVRT, SAAWR | Supraventricular tachycardia |
| SB | Sinus Bradycardia (SB) | Sinus bradycardia |
| SR | Sinus Rhythm (SR), Sinus Irregularity (SA) | Normal sinus rhythm |

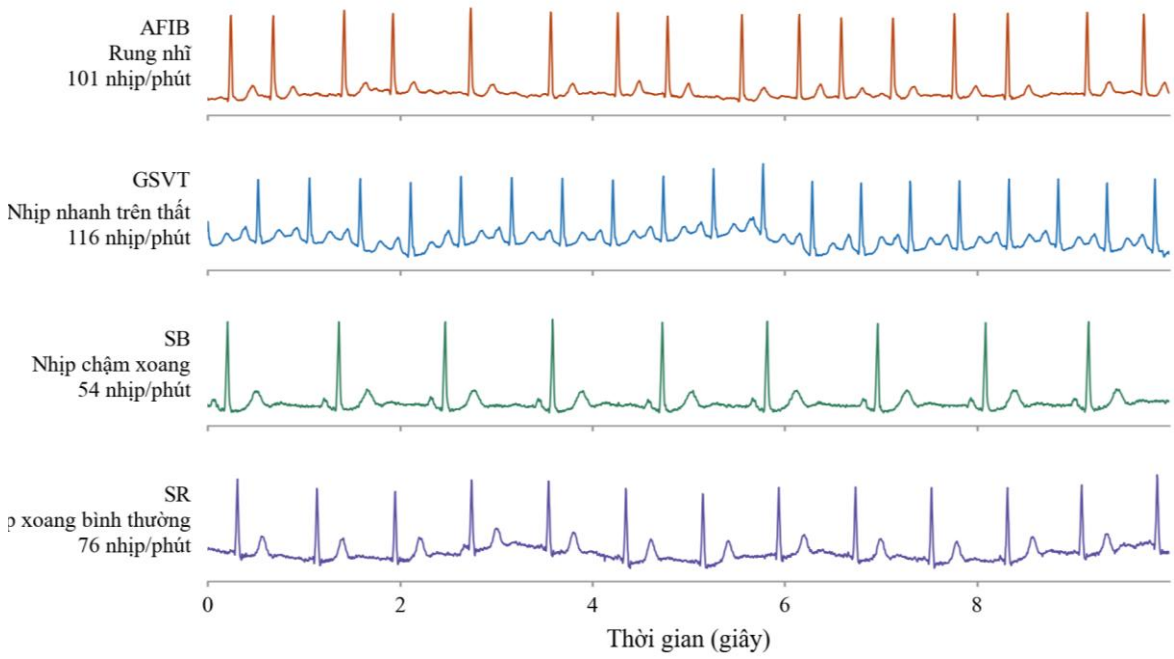



Figure 2.1. Typical ECG waveforms of the four rhythm classes; each trace is one 10-second record from the test set.

Table 2.2. Records per class in the Chapman dataset

| Class | Training | Validation | Test | Total |
|---|---|---|---|---|
| AFIB | 5,273 | 1,130 | 1,130 | 7,533 |
| GSVT | 4,051 | 868 | 869 | 5,788 |
| SB | 8,355 | 1,791 | 1,791 | 11,937 |
| SR | 5,519 | 1,183 | 1,183 | 7,885 |
| Total | 23,198 | 4,972 | 4,973 | 33,143 |

The cross-dataset test set is the Georgia 12-lead ECG Challenge Database [2], collected at Emory University, USA, and released for the PhysioNet/CinC Challenge 2020. It was acquired with an entirely different recording system and shares no records with the

training set, so it constitutes a test of far generalization. Its records are likewise 12-lead ECGs of 10 s at 500 Hz, but labelled with SNOMED-CT codes; records mapping unambiguously onto one of the four classes of Table 2.1 were kept, yielding 5,459 records. Georgia is used for evaluation only and takes no part in training, so the results on it reflect the model's behaviour on data from an unseen acquisition system.

## 2.2. OVERVIEW OF CONVOLUTIONAL NEURAL NETWORKS

### 2.2.1. Neural, convolutional and one-dimensional networks

An artificial neural network consists of layers of interconnected processing units. Each unit takes a weighted linear combination of its inputs, adds a bias and applies a non-linear function; all weights are learned from data. In the fully connected form, each unit connects to every unit of the previous layer, so the weight count grows very rapidly with input size: for a 2,500-sample sequence the first layer alone would need several million parameters. This organization also ignores a key signal property: an abnormal waveform means the same early or late in the record.

The convolutional neural network (CNN) overcomes this through two principles: local connectivity, where each unit sees only a narrow input window, and weight sharing, where the same weights slide along the whole signal. The parameter count then depends only on window size and channel count, not on signal length, and the network becomes translation invariant. Stacking convolutional layers builds a feature hierarchy: shallow layers learn elementary waveforms, deeper layers combine them into more complex patterns.

The one-dimensional network (1D-CNN) applies to sequential signals, the kernel sliding along a single time axis. Compared with turning the ECG into a two-dimensional image and applying an image CNN [6], it works directly on the sample sequence and so costs far less computation. Kiranyaz et al. [3] showed that a shallow 1D-CNN can classify heartbeats in real time with competitive accuracy; later studies extended this to large deep networks reaching cardiologist-level performance [4], or took the opposite path of compacting the model for wearables [5].

### 2.2.2. The 1D-CNN model layer by layer

**The Convolution Layer** slides a weight kernel along the input sequence, computing at each position the weighted sum over the current window. For an input sequence x with C channels, a kernel of size K and a bias b, the output of channel o at position t is:

$$y_o(t) = \sum_c \sum_k x_c(t + k - P) \cdot w_{o,c,k} + b_o \quad (2.1)$$

where P is the number of samples padded at each end. As the kernel slides over every position t, the same weights are reused across the signal. A layer has C × O × K + O parameters, independent of signal length. With P = (K − 1)/2 and K odd, the output length equals the input length, so the data size changes only at pooling layers.

**The Pooling Layer** compacts the sequence by dividing it into non-overlapping windows and retaining one representative value for each window. This thesis employs max-pooling with a window of size S and a stride equal to S:

$$p_o(i) = \max(y_o(i \cdot S), \dots, y_o(i \cdot S + S - 1)) \quad (2.2)$$

Max-pooling keeps the strongest response in each window — the clearest evidence that a feature is present — while discarding its exact location, which suits rhythm classification. It has no learnable parameters but shortens the sequence by a factor of S, reducing the load of later layers.

**The ReLU activation function** introduces non-linearity into the network; without it, a stack of linear layers would remain equivalent to a single linear transformation. The most widely used function is the ReLU (Rectified Linear Unit):

$$\mathrm{ReLU}(z) = \max(0, z) \quad (2.3)$$

ReLU is favoured because its derivative is 1 for every positive input, so it does not cause vanishing gradients, and because in hardware it is a single sign comparison. Note that ReLU discards the whole negative part. For images this loses little, since pixel intensities are non-negative; for the ECG the negative part carries real physiological information — the Q and S waves are negative deflections of the QRS complex. Where

the activation is placed is therefore a design choice, not an automatic step after every convolutional layer.

**Global Average Pooling (GAP)** replaces flattening of the whole feature map by the mean of each channel over the time axis, reducing each channel to one number:

$$g_o = \left(\frac{1}{L}\right) \cdot \sum_i p_o(i) \quad (2.4)$$

This [8] sharply cuts the parameters of the classification layer: flattening gives it channels × remaining length inputs, whereas GAP gives it only channels. GAP also makes the model less dependent on absolute feature position and less sensitive to local noise.

**The Fully-Connected Layer (FC)** is the final layer, mapping the resulting feature vector onto a score for each classification class:

$$z_j = \sum_o g_o \cdot W_{j,o} + b_j \quad (2.5)$$

where j ranges over the classes. Each z[j] is a raw score, not yet normalized into a probability.

**Argmax** selects the class with the highest score as the predicted result:

$$\text{lớp dự đoán} = \text{argmax}_j\, z_j \quad (2.6)$$

In training, the scores z pass through softmax to form a probability distribution for the loss. Softmax is monotonically increasing and so does not change the ordering of the scores; at inference only the highest-scoring class matters, so softmax can be dropped and z compared directly. This is an important hardware simplification, since softmax needs exponentiation and division — both expensive in integer arithmetic.

## 2.3. MODEL TRAINING

**The learning mechanism:** Training seeks the weights that bring the network's predictions closest to the true labels, repeating three steps on each batch. The forward pass propagates data through the network to obtain the scores. The loss compares prediction with true label and measures the error. Backpropagation applies the chain rule to obtain the gradient of the loss for each weight, and the optimizer updates the weights accordingly.

One pass over the training set is an epoch; the model is evaluated on the validation set after each epoch and only the best checkpoint kept, to avoid selecting a state that has merely memorized the training data.

**The cross-entropy loss:** For multi-class problems the standard loss is cross-entropy. The FC scores z become probabilities via softmax, and the negative log of the probability of the correct class is taken:

$$L = -\log\left(\frac{\exp(z_y)}{\sum_j \exp(z_j)}\right) \tag{2.7}$$

where y is the correct class index. The function penalizes a low probability on the correct class very heavily, so the model must predict correctly and with appropriate confidence. Combined with softmax, the gradient in z also reduces to predicted probability minus label — cheap and numerically stable.

**The Adam optimization algorithm.** The weight-update algorithm employed is Adam [7]. Instead of applying the same learning rate to every parameter, as in basic gradient descent, Adam maintains two running averages for each parameter: the average of the gradient (the first moment) and the average of the squared gradient (the second moment):

$$m(t) = \beta_1 \cdot m(t-1) + (1 - \beta_1) \cdot g(t) \tag{2.8}$$

$$\theta(t) = \theta(t-1) - \eta \cdot \frac{\hat{m}(t)}{\left(\sqrt{\hat{v}(t)} + \varepsilon\right)} \tag{2.9}$$

where g(t) is the gradient at step t, η the learning rate, and $\hat{m}$ and $\hat{v}$ the bias-corrected moments. The first moment acts as inertia across regions of oscillating gradients; the second normalizes the step to the recent gradient magnitude, so parameters with small gradients are still updated enough and those with large gradients do not overshoot. Adam was chosen for its rapid convergence and low sensitivity to the initial learning rate — useful when retraining repeatedly during model compression.

**Structured pruning:** A trained model usually holds many parameters that contribute very little. Pruning removes them to compact the model [9]. Two approaches exist.

Unstructured pruning removes individual weights, giving a sparse matrix; the compression ratio is high but yields a real benefit only if the hardware can skip zeros. Structured pruning removes a whole filter with its output channel, genuinely shrinking the network: fewer channels mean fewer multipliers and less memory, with no special hardware support. For an FPGA implementation the structured approach is therefore preferable.

Filter importance is judged by the L1 norm [11], the sum of absolute weights in the filter:

$$S(o) = \sum_c \sum_k |w_{o,c,k}| \tag{2.10}$$

A filter with small S responds weakly to every input and contributes little to the decision, so it is removed first. Unlike gradient-based criteria such as the Taylor approximation [10], the L1 norm needs only the weights and is independent of any data batch. Removing channels lowers accuracy; the model is then fine-tuned at a small learning rate so the remaining channels compensate.

## 2.4. QUANTIZATION

After training, weights and intermediate values are 32-bit floating-point (float32). On an FPGA this format is costly in three ways: a floating-point multiplier uses more resources than an integer one of the same width, needs more cycles, and each value takes four times the memory of an INT8 value. Quantization converts the model to narrow integers, trading some accuracy for a large cut in resources and energy [15].

**Symmetric quantization:** The idea is to represent a real number as an integer times a common scale factor [12]. For a range symmetric about zero and 8 bits, the integer lies in [−127, 127] and the conversion is:

$$q = \text{clamp}\left(\text{round}\left(\frac{r}{s}\right), -127, 127\right) \tag{2.11}$$

where r is the real value, q the stored integer, and s the scale factor set by the maximum absolute value of the quantity. The clamp keeps the result representable, clipping a few outliers in exchange for better resolution over the remaining values.

**Accumulation and rescaling.** When input and weights are both INT8, their product needs 16 bits and many products are accumulated in an INT32 register to avoid overflow. The accumulated result is thus on a far larger scale than the 8-bit input of the next layer and must be brought back — the role of rescaling. In the general scheme rescaling is a multiplication by a real coefficient plus rounding, so each layer needs an extra multiplier.

**Power-of-two quantization.** A variant of particular interest for hardware is to constrain the scale factor to be a power of two, $s = 2^{-nb}$ [14]. Division by the scale factor then becomes a right shift by nb bits:

$$\mathrm{nb} = \left\lfloor \log_2 \left( \frac{127}{|r|_{\max}} \right) \right\rfloor \tag{2.12}$$

$$\mathrm{out} = \mathrm{clamp}\left( \left( \mathrm{acc} + 2^{\mathrm{nb}-1} \right) \gg \mathrm{nb}, -127, 127 \right) \tag{2.13}$$

The immediate benefit is that rescaling no longer requires a multiplier: a shifter and an adder suffice. The additive term $2^{nb-1}$ in (2.13) rounds to the nearest integer rather than truncating the fraction — a small hardware detail that still affects accuracy, since truncation gives a one-sided error that accumulates across layers. The price is that the scale factor is rounded down to the nearest power of two, wasting at most half the representable range.

**Two ways to reach integer arithmetic.** There are two ways to move a model to integer arithmetic. Post-training quantization (PTQ) merely probes the value ranges on a small dataset and converts, without retraining. Quantization-aware training (QAT) inserts fake quantization during training so the network adjusts its weights to the rounding error. The difficulty with QAT is that rounding has a zero derivative almost everywhere, so gradients cannot propagate; the standard remedy is the straight-through estimator [13], which treats rounding as the identity when differentiating. At the cost of extra training, QAT generally preserves accuracy better, especially at low bit widths or with a constrained scale factor.

## 2.5. HARDWARE ARCHITECTURE

**Model versus execution architecture.** A model specifies the computation but not how the hardware should be arranged to perform it. The same network admits many architectures, differing in how many multiplications run concurrently and how data are reused. Three quantities govern the choice: the total multiply–accumulate count, the total weight storage, and how much the data shape changes between layers.

For large convolutional networks the weights exceed on-chip memory and sit in external memory, so the bottleneck is bandwidth, not compute [19]. For a network small enough to keep all weights on chip the problem changes: with no bandwidth bottleneck, what must be optimized is logic and energy. A further trait of one-dimensional networks with pooling is that the data shape varies markedly — the first layer handles a long sequence with few channels, the last a short one with many — so the dimension chosen for parallelism must be the one varying least, or the hardware idles in some layers.

**The axes of architectural classification.** According to Sze et al. [18], accelerators differ along two independent axes. The first is how parallelism is exploited: temporal architectures share one central computational block for the whole computation, saving resources but limiting throughput; spatial architectures spread the processing units into an array with direct data transfer, giving high throughput but costing resources and being efficient only when each layer fills the array.

The second axis is how layers map onto hardware. A single-block time-shared architecture uses one convolution block, loading each layer's weights in turn and reusing the block. A fully mapped architecture dedicates hardware to each layer and runs them concurrently in pipelined fashion: throughput is very high, but resources grow with the layer count and the slowest layer dictates the system speed [20].

**The architecture selected for this thesis.** Against the characteristics of the problem, these axes lead to a single-block time-shared architecture running the layers sequentially, parallel in two dimensions: output channels and weights within the kernel.

# CHAPTER 3: DESIGN AND IMPLEMENTATION OF THE CNN

This chapter presents the design and implementation of the CNN accelerator for arrhythmia classification, from the software model to the hardware core and the host interface. It has three parts matching the design flow: data processing and training in PyTorch, ending with quantization and extraction of the INT8 weights (Section 3.1); the hardware architecture in Verilog — convolution–pool block, parallel engine, output classifier and controller (Section 3.2); and integration of the core with the Avalon-MM bus and the JTAG-to-Avalon bridge for host control (Section 3.3). Figure 3.1 summarizes the flow.

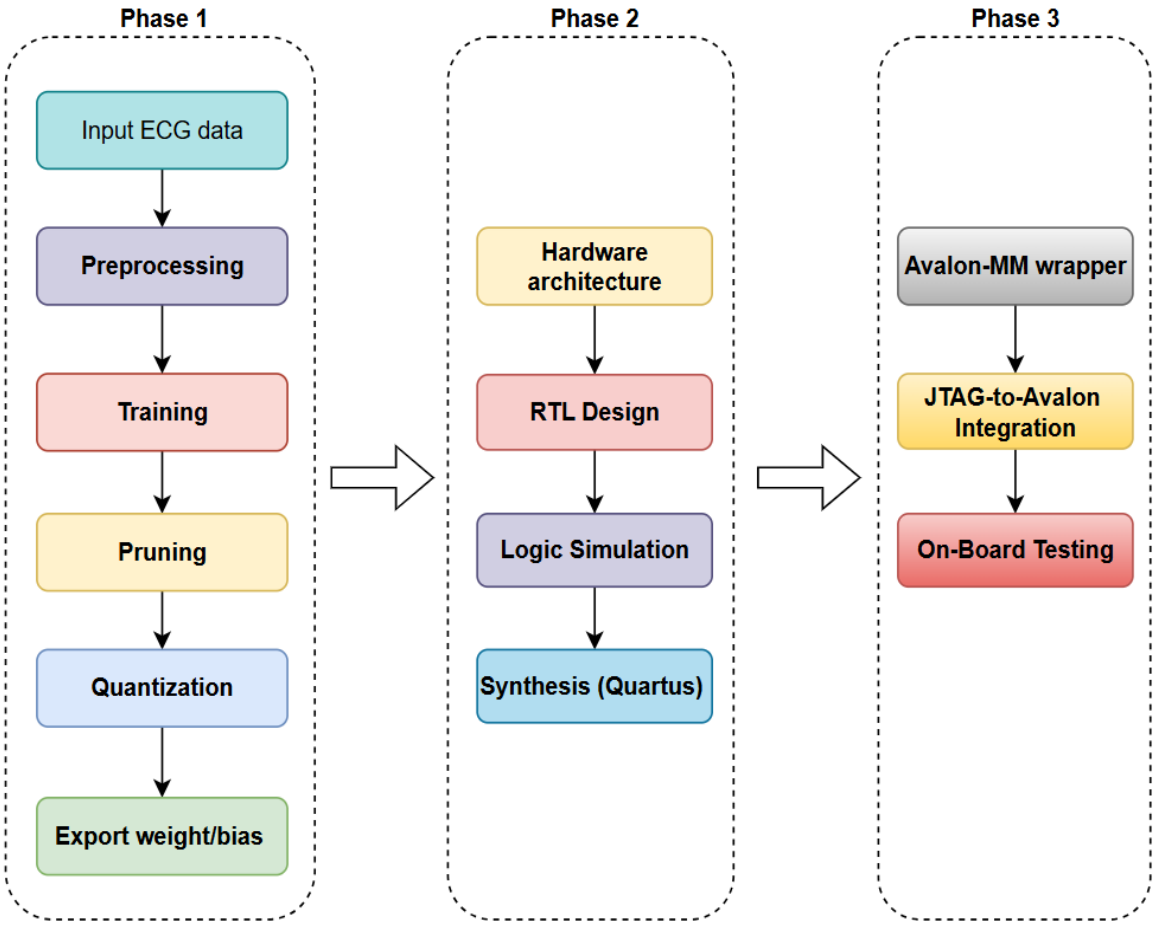



Figure 3.1. Design flow from software model to FPGA demonstration.

## 3.1. DATA PROCESSING AND MODEL TRAINING WORKFLOW

The software stage aims to produce a compact, accurate INT8 weight set formatted for direct loading into the hardware, by the sequential steps of Figure 3.2, matching the four subsections below.

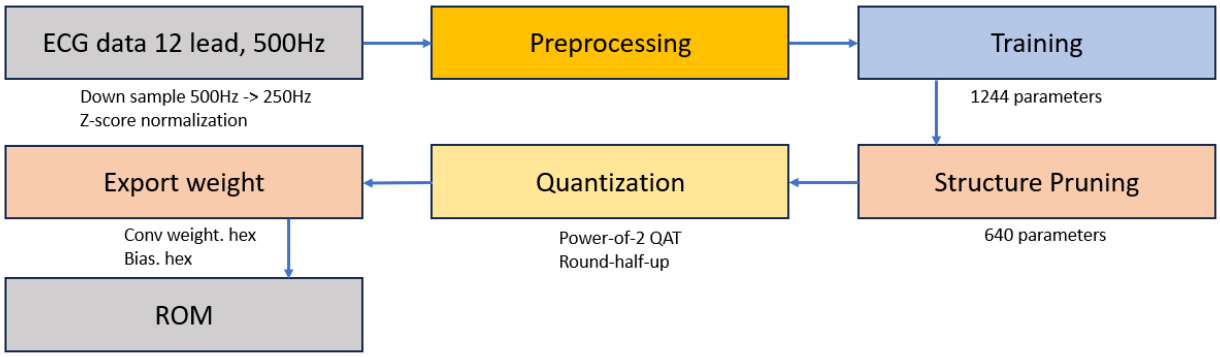



Figure 3.2. Software flow from raw data to the hardware weight set.

### 3.1.1. Data processing

The main dataset for training and evaluation is Chapman [1]: 12-lead ECG records of 10 s at 500 Hz, labelled at record level with SNOMED-CT rhythm codes. Its full name and provenance appear in Section 2.1.2.

The detailed rhythm codes of the original data are grouped into the four target groups of Chapter 2, by rules based on the electrophysiology of each group (Table 2.1). The same rules apply to the cross-dataset test set of Chapter 4, keeping labels consistent across sources — a prerequisite for meaningful comparison.

From the 12-lead record, only lead II is extracted as a single-channel input, for the reasons set out in Chapter 2. The signal then passes through two preprocessing steps:

– Step 1: Resampling from 500 to 250 Hz, reducing the 10-second window from 5,000 to 2,500 samples.

– Step 2: Per-record Z-score normalization, bringing every record onto a common amplitude scale.

The data are divided in the ratio **70/15/15** into the training, validation and test sets, following the principle of **patient independence**: records of the same patient never appear

in two sets. This prevents leakage that would inflate the reported accuracy and reflects real deployment, where the model meets unseen patients. Table 3.1 gives the size of each set.

Table 3.1. Dataset partitioning.

| Set | Records | Proportion | Role |
|---|---|---|---|
| Training | 23,198 | 70 % | Weight updating |
| Validation | 4,972 | 15 % | Best-model selection, early stopping |
| Test | 4,973 | 15 % | Final evaluation, not used during training |
| Total | 33,143 | 100 % | |

### 3.1.2. Model training

The model is a one-dimensional CNN of four convolutional layers, each followed by max-pooling, ending in global average pooling and a fully connected layer. Before pruning it has the channel configuration (4, 8, 8, 16) and **1,244 parameters**, as presented in Figure 3.3.

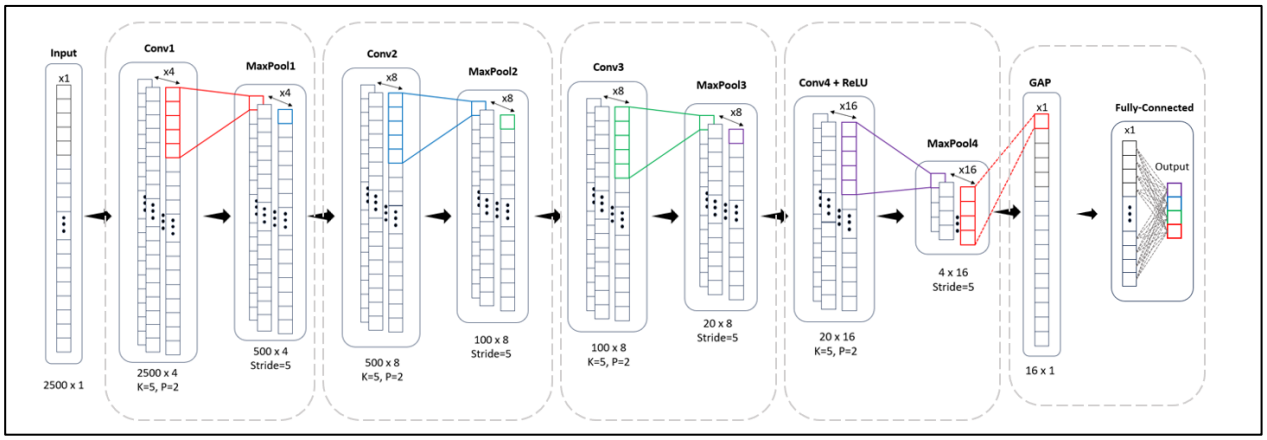



Figure 3.3. CNN architecture before pruning (1,244 parameters).

The model is trained with weights represented as 32-bit floating-point numbers, using the cross-entropy loss and the Adam optimizer. The training loop comprises four steps:

– Step 1: Forward pass: a batch is propagated through the network to obtain the prediction.

– Step 2: Loss: the prediction is compared with the true label by cross-entropy.

– Step 3: Backpropagation: the derivative of the loss with respect to each weight is computed.

– Step 4: Weight update: the Adam optimizer adjusts the weights in the direction that reduces the loss.

After each epoch the model is evaluated on the validation set and only the weights with the best validation accuracy are kept, to avoid a model that has memorized the training set.

The trained model is still redundant for FPGA deployment. To compact it, **this thesis applies structured channel pruning**: whole filters of low importance are removed instead of setting single weights to zero as in unstructured pruning. The distinction is decisive for hardware — only structured pruning truly reduces the tensor dimensions and hence the multipliers and memory needed; unstructured pruning gives a sparse matrix for which full resources must still be provided.

Filter importance is assessed by the L1 norm [11], the sum of the absolute weights in the filter. Filters with a small L1 norm contribute weakly to the output features and go first. The procedure is:

– Step 1: Compute the L1 norm of every filter in each convolutional layer.
– Step 2: Rank the filters by decreasing L1 norm.
– Step 3: Keep the desired number of filters and remove the rest with all related channels.

Table 3.2. Channels per layer before and after structured pruning.

| Layer | Initial | After pruning | Action |
|---|---|---|---|
| Conv1 | 4 | 4 | Unchanged |
| Conv2 | 8 | 4 | Remove the 4 filters with the smallest L1 norm |
| Conv3 | 8 | 8 | Unchanged |
| Conv4 | 16 | 8 | Remove the 8 filters with the smallest L1 norm |
| FC | 16 | 8 | Narrowed in accordance with Conv4 |

Pruning discards part of the learned information, so accuracy decreases temporarily. The model is therefore fine-tuned in two phases with a decreasing learning rate:

– Phase 1: 30 epochs at a learning rate of $10^{-3}$ — rapid recovery after pruning.
– Phase 2: 20 epochs at a learning rate of $10^{-4}$ — fine adjustment and stable convergence.

The result has the channel configuration (4, 4, 8, 8) and 640 parameters, 48.6% fewer than the initial model; this is the configuration implemented in hardware.

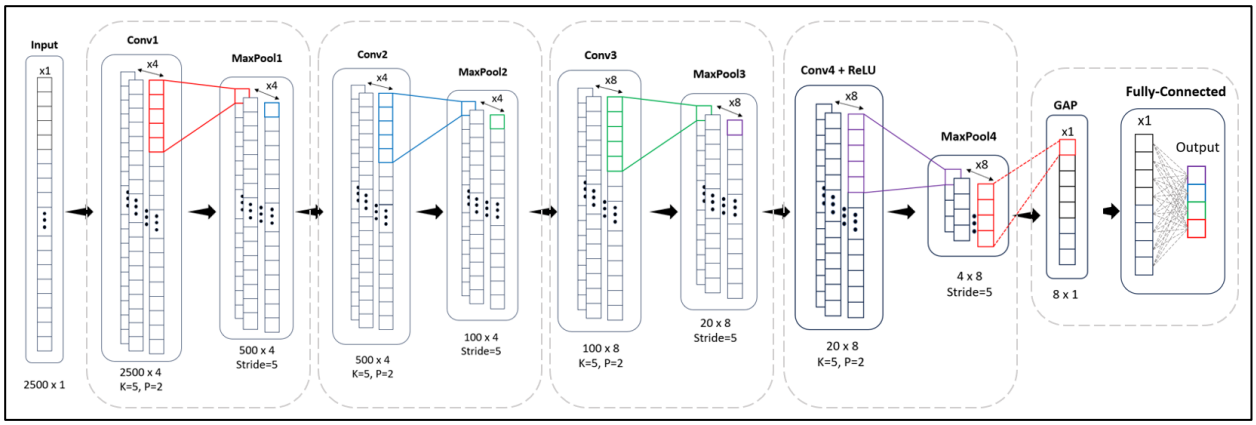



Figure 3.4. CNN after pruning (640 parameters) — the hardware configuration.

Table 3.3. Layer configuration and data-size transformations of the pruned model.

| Layer | Input channels | Output channels | K | P | Max-Pool | ReLU | Input | Output | Parameters |
|---|---|---|---|---|---|---|---|---|---|
| Conv1 | 1 | 4 | 5 | 2 | /5 | No | 1 × 2500 | 4 × 500 | 24 |
| Conv2 | 4 | 4 | 5 | 2 | /5 | No | 4 × 500 | 4 × 100 | 84 |
| Conv3 | 4 | 8 | 5 | 2 | /5 | No | 4 × 100 | 8 × 20 | 168 |
| Conv4 | 8 | 8 | 5 | 2 | /5 | Yes | 8 × 20 | 8 × 4 | 328 |
| GAP | 8 | 8 | — | — | /4 | — | 8 × 4 | 8 × 1 | 0 |
| FC | 8 | 4 | — | — | — | — | 8 | 4 | 36 |
| Total | | | | | | | | | 640 |

Each convolutional layer has (input channels × output channels × K) + output channels parameters; Conv4, for example, has 8 × 8 × 5 + 8 = 328.

### 3.1.3. Weight quantization

The pruned model is converted from floating point to 8-bit signed integers (INT8) by quantization-aware training with power-of-two scaling. Quantization represents a floating-point tensor as 8-bit integers with a scale factor. When that factor is a power of two, multiplying by it degenerates into a bit shift — on an FPGA merely a rewiring of bits, costing almost nothing, whereas a general factor needs one DSP block per multiplication. This is the whole motivation.

The bit-shift coefficient of each tensor $T$ is chosen according to the maximum absolute value of that tensor:

$$\text{shift}_{\text{bits}} = \left\lfloor \log_2 \left( \frac{127}{\max|T|} \right) \right\rfloor \tag{3.1}$$

where $max|T|$ is the maximum absolute value of tensor T and 127 is the largest value representable in signed INT8. The formula takes the largest power of two that still preserves the full value range of the tensor after scaling, so that no value is clipped at the boundary.

The INT8 pipeline is identical in the Python model and the circuit, in four steps.

Step 1 — Input quantization. The Z-score normalized ECG signal is scaled by $2^{input\_shift}$ then rounded and clamped to the INT8 range:

$$x_{\mathrm{q}} = \mathrm{clamp}(\mathrm{round}(x \cdot 2^{\mathrm{input_{shift}}}), -127, 127) \tag{3.2}$$

Step 2 — Weight quantization. Each layer has its own bit-shift coefficient, computed by (3.1) on the weight tensor of that layer:

$$w_{\mathrm{q}} = \mathrm{clamp}(\mathrm{round}(w \cdot 2^{w_{\mathrm{shift}}}), -127, 127) \tag{3.3}$$

Step 3 — Convolution and bias. Products of INT8 pairs accumulate in a 32-bit accumulator, to which the scaled bias is added:

$$\mathrm{acc} = \sum(x_{\mathrm{q}} \cdot w_{\mathrm{q}}) + \mathrm{bias_{scaled}} \tag{3.4}$$

Step 4 — Rescaling. The 32-bit result returns to the INT8 range as input for the next layer:

$$\mathrm{out} = \mathrm{clamp}\left(\mathrm{round-half-up}\left(\frac{\mathrm{acc}}{2^{\mathrm{nb}}}\right), -127, 127\right) \tag{3.5}$$

Section 2.4 gives the rationale: the 32-bit accumulator is on a far larger scale than the INT8 range of the next layer and must be brought back. With power-of-two scaling this is only a right shift of *nb* bits with rounding, and needs no multiplier.

The bias is scaled according to the formula $\mathrm{bias_{scaled}} = \mathrm{round}(b \cdot 2^{\mathrm{nb}})$ and stored as INT32. The bias must carry the accumulator scale — not the weight scale — for the addition in (3.4) to be dimensionally valid, as is standard in integer quantization [12][15]. The values of *w_shift* and *nb* after calibration on the training set are given in Table 3.3.

Table 3.4. Power-of-two quantization parameters for each layer.

| Layer | w_shift (weight scale) | nb (rescaling) | Remarks |
|---|---|---|---|
| Conv1 | 6 | 8 | nb = input_shift + w_shift = 2 + 6 |
| Conv2 | 7 | 7 | INT8 range, so nb = w_shift |
| Conv3 | 6 | 6 | INT8 range, so nb = w_shift |
| Conv4 | 7 | 7 | INT8 range, so nb = w_shift |
| FC | 7 | 0 | No rescaling |

The second difference from earlier shift-based designs — typically Liu [16], which truncates the fraction — is that rescaling here rounds to the nearest integer, not downwards. Rounding adds half the unit of the shifted-out bit before the shift:

$$\text{round} - \text{half} - \text{up}(\text{acc}) = \left(\text{acc} + 2^{\text{nb}-1}\right) \gg \text{nb} \tag{3.6}$$

Section 2.4 gives the reason: an arithmetic right shift always rounds towards negative infinity, so the error is one-sided and accumulates across layers, whereas round-half-up is symmetric about zero. The cost is one constant addition before the shift.

### 3.1.4. Weight extraction

The last software step converts weights and biases into hexadecimal text files with the layout the hardware expects.

For the convolutional layers, the weights are arranged in the order **output channel outermost, input channel innermost**, and the five weights of an (output channel, input channel) pair are **packed into a single 40-bit word** comprising five bytes. This matches the hardware: each block handles one output channel and needs, each cycle, the five weights of one input channel — one 40-bit word in a single access.

The 40-bit packing is far more economical than storing single bytes and removes the need for byte assembly at run time.

The weights of the fully connected layer are stored with **the output channels as rows**, forming 4 rows × 8 columns for the sequential multiply–accumulate loop of the output classifier. The software stage thus ends with an INT8 model bit-accurate with the hardware and a weight set ready to load.

Table 3.5. Weight files loaded into the hardware ROMs.

| File | Words | Width | Contents |
|---|---|---|---|
| conv1_w.hex | 4 | 40 bit | 4 output channels × 1 input channel, 5 weights per word |
| conv2_w.hex | 16 | 40 bit | 4 output channels × 4 input channels |
| conv3_w.hex | 32 | 40 bit | 8 output channels × 4 input channels |
| conv4_w.hex | 64 | 40 bit | 8 output channels × 8 input channels |
| conv_bias.hex | 32 | 32 bit | INT32 bias, address = output channel × 4 + layer index |
| fc_weights.hex | 32 | 8 bit | FC weights, address = output channel × 8 + input |
| fc_bias.hex | 4 | 32 bit | INT32 FC bias, already scaled by 2^w_shift |

## 3.2. DESIGN OF THE CNN SYSTEM

The hardware is designed around the architecture of **a single shared computational engine, time-multiplexed**. Instead of unrolling the four convolutional layers into four hardware blocks, one engine of eight processing elements is reused sequentially for all four under a finite state machine. This suits a model of only 640 parameters: fully unrolled, the hardware for Conv1 (one input channel) would idle most of the time, whereas reusing the engine keeps area low while meeting the latency needed for continuous monitoring.

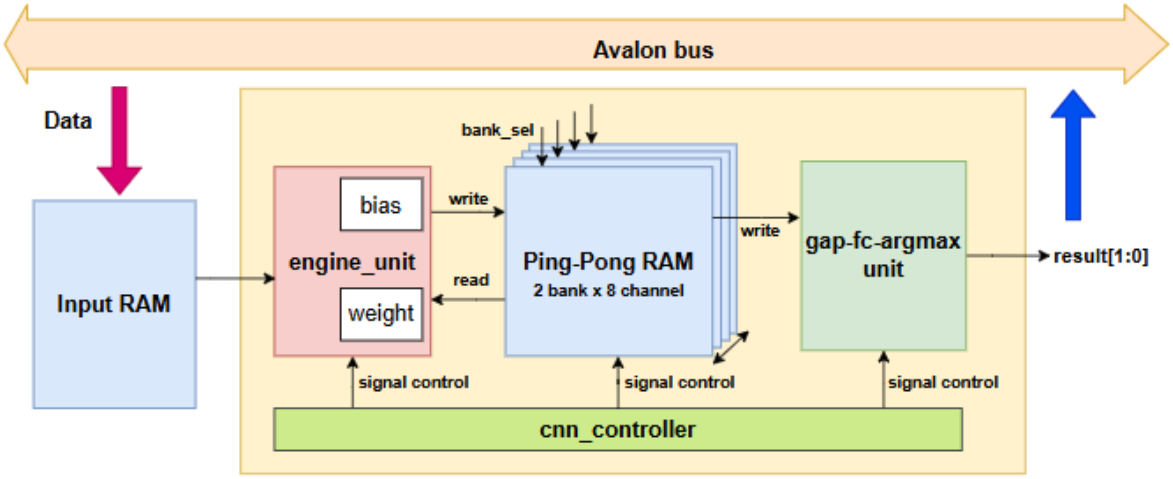



Figure 3.5. Architecture of the CNN system of this work.

Table 3.6 lists the hardware modules, their roles and where they are presented.

Table 3.6. List of RTL modules in the design.

| Module | Role |
|---|---|
| mul_add | Five multipliers and a three-stage adder tree giving the convolution sum |
| accumulate_rescale | Multi-channel accumulation, bias, rescaling, saturation and ReLU |
| pool | Max-pooling over a window of 5 |
| convolution-pool unit | Wraps the three blocks into one complete output channel |
| engine unit | Eight parallel convolution-pool units and the sliding window |
| gap_unit / fc_unit / argmax_unit | Global average pooling, fully connected, maximum search |
| cnn_controller | Finite state machine coordinating the entire system |
| ping_pong_sram | Inter-layer feature-map buffer, two banks |
| input_sram | Input memory, 2500 × 8 bit |
| core | Wraps the whole computational core, bus-independent |

### 3.2.1. Design of the convolution-pool unit

The **convolution-pool unit** block is the elementary computational unit and handles one output channel. Each clock cycle it takes a window of 5 samples with the 5 matching weights, performs the one-dimensional convolution, rescales to INT8 and applies max-pooling. It has three sub-blocks, for clarity and separate verification; the division is structural and does not alter the arithmetic. They are mul_add (multiplications and adder tree), accumulate_rescale (accumulation and rescaling) and pool. Figure 3.6 shows them.

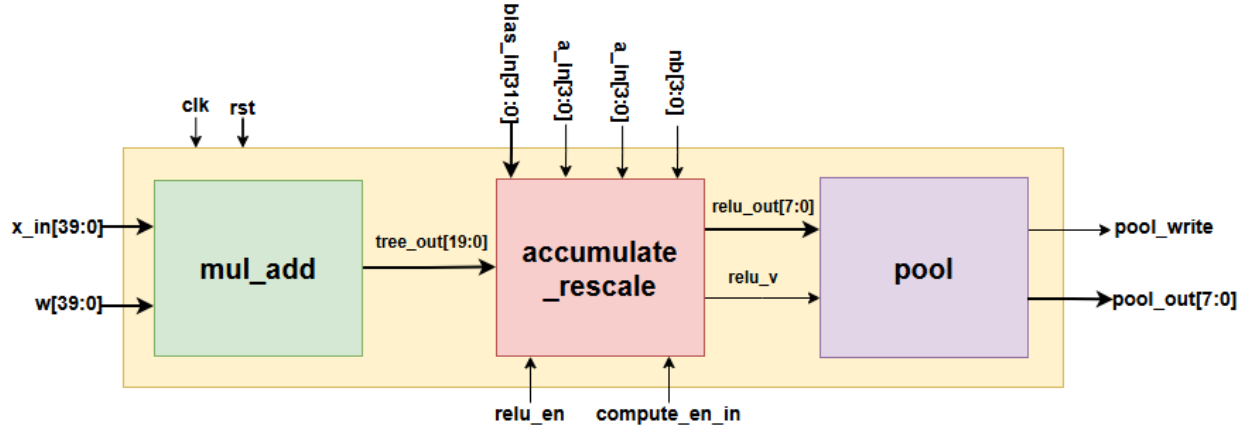



Figure 3.6. The convolution-pool unit and its three sub-blocks.

Table 3.7. Port interface of the convolution-pool unit.

| Port | Direction | Width | Description |
|---|---|---|---|
| clk | in | 1 | System clock |
| rst | in | 1 | Reset signal, active high |
| x_in | in | 40 | The five samples of the window, packed as 5 × 8 bit |
| w | in | 40 | The five weights, packed as 5 × 8 bit |
| bias_in | in | 32 | Scaled INT32 bias |
| a_in | in | 4 | Input-channel counter, delayed by 5 cycles |
| in_ch | in | 4 | Number of input channels of the current layer |
| compute_en_in | in | 1 | Pipeline enable, delayed by 5 cycles |
| nb | in | 4 | Number of shift bits used in rescaling |
| relu_en | in | 1 | Enables ReLU; equal to 1 only in Conv4 |
| pool_rst | in | 1 | Clears the pooling counter on a layer transition |
| pool_write | out | 1 | Strobe: a valid pooled value is ready to write |
| pool_out | out | 8 | INT8 value after max-pooling |

Internally the block is a multi-stage pipeline with register layers between stages, which shortens the combinational path between registers and raises the maximum frequency. Table 3.8 describes each stage.

Table 3.8. Pipeline stages of the convolution-pool unit.

| Stage | Sub-block | Operation | Bit |
|---|---|---|---|
| S1 | mul_add | Five signed 8 × 8 bit multiplications | 16 |
| S2 | mul_add | Pairwise addition: *sum01 = p0 + p1, sum23 = p2 + p3* | 17 |
| S3 | mul_add | Further addition: *sum0123 = sum01 + sum23* | 18 |
| S4 | mul_add | Addition of the fifth branch, producing *tree_out* | 20 |
| S5 | accumulate_rescale | Multi-channel accumulation, bias and rounding constant pre-added | 32 |
| S_bias | accumulate_rescale | One cycle of delay, then the complete accumulated value is latched | 32 |
| S6 | accumulate_rescale | Arithmetic right shift by nb | 32 |
| S7 | accumulate_rescale | Saturation to the range [−127, 127] | 8 |
| S8 | accumulate_rescale | ReLU, effective only in Conv4 | 8 |
| S9 | pool | Max-pooling over a window of 5 | 8 |

**Stage S1: multiplication.** Five sample–weight pairs are multiplied at once by five signed 8 × 8 bit multipliers giving 16-bit results. Both operands are two's complement, so the multiplication is signed.

**Stages S2 to S4: the adder tree.** The five products are summed through three registered stages rather than in one cycle. The result tree_out is the convolution sum of one output position for **one input channel**.

**Stage S5: multi-channel accumulation.** In the multi-channel layers Conv2 to Conv4, the convolution sums of the individual input channels must be accumulated into a complete result, over exactly in_ch consecutive cycles. Here **the bias and the rounding constant are merged into the initialization term** of the accumulator instead of being added at later stages.

The transformation is **exactly bit-accurate** with the original expression (acc + bias + round) >> nb, by associativity of addition, and all bounds lie safely inside the 32-bit signed range, so overflow is impossible.

**Stage S_bias: latching the result.** This stage performs no addition; it merely delays the valid signal by one further cycle and latches the accumulated value.

**Stage S6: rescaling shift.** The 32-bit value is right-shifted by nb bits, the rounding constant merged in at S5.

**Stages S7 and S8: saturation and ReLU.** The shifted result is clamped to the valid INT8 range: above 127 becomes 127, below −127 becomes −127. In Conv4 alone, stage S8 sets negative values to 0, implementing ReLU; other layers pass through unchanged.

**Stage S9: max-pooling.** The pool block holds a comparator and a counter from 0 to 4. The first valid value goes straight into the maximum register; later values update it only if larger. On the fifth the block asserts pool_write with the result and resets the counter. The counter is gated by compute-enable so it ignores the spurious values of the window priming phase.

### 3.2.2. Design of the engine unit

The **engine unit** block places **eight convolution-pool units in parallel** to compute eight output channels at once, with the full data-supply mechanism around them: sliding register window, branch selector, delay chain, weight store and selective write ports. This block takes most of the multiplier resources, five multipliers for each of eight channels. Figure 3.7 shows the organization.

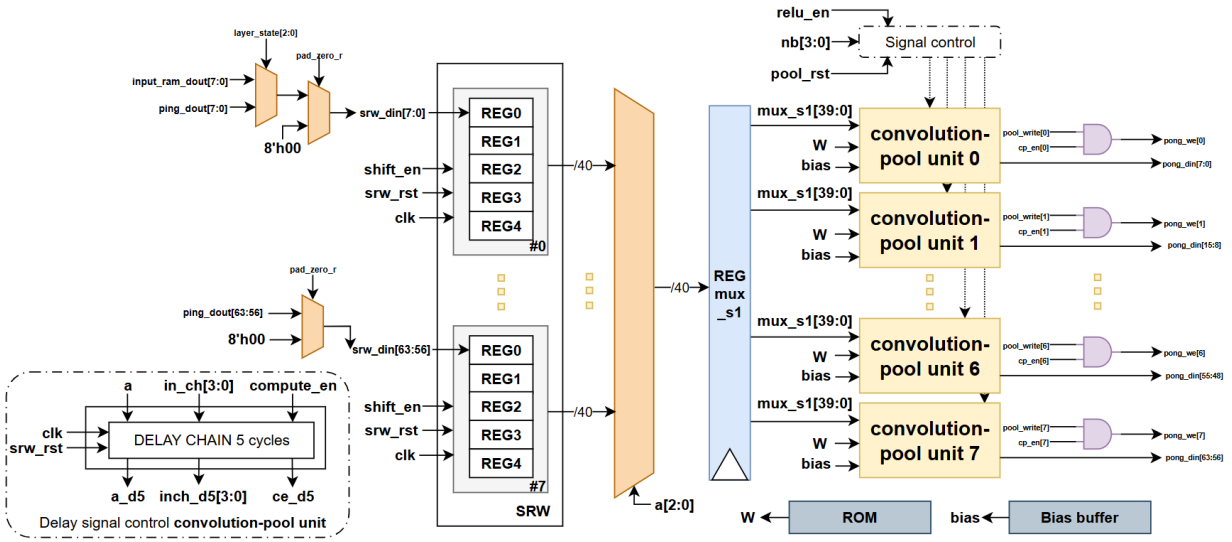



Figure 3.7. Engine unit flow diagram.

Table 3.9. Port interface of the engine unit.

| Port | Direction | Width | Description |
|---|---|---|---|
| clk, rst | in | 1 | Clock and reset signals |
| a | in | 4 | Input-channel counter, from 0 to in_ch − 1 |
| in_ch | in | 4 | input channels of the current layer |
| in_len | in | 12 | Input length: 2500 / 500 / 100 / 20 |
| shift_en | in | 1 | Window shift command, 1 when a is final |
| srw_rst | in | 1 | Clears the sliding on a layer transition |
| compute_en | in | 1 | Compute enable, 0 during the priming phase |
| nb | in | 4 | shift bits used for rescaling in the layer |
| relu_en | in | 1 | Enables ReLU, in Conv4 only |
| cp_en | in | 8 | Mask identifying the active output channels |
| layer_state | in | 3 | Layer currently executing, used to select the weight ROM |
| pool_rst | in | 1 | Clears the pool counter at a layer transition |
| input_sram_dout | in | 8 | Data input memory, used by Conv1 only |
| ping_dout | in | 64 | Data from the Ping bank, 8 channels packed |
| sram_rd_addr_in | in | 12 | Current output position by the controller |
| pong_din | out | 64 | Data written to the Pong bank, 8 channels |
| pong_we | out | 8 | Per-channel write enable |
| sram_rd_addr | out | 12 | Read address sent to the memory |

**The sliding register window.** Each input channel has a shift register of 5 cells of 8 bits. Data flow through the window cycle by cycle: on each shift the newest sample enters cell 0 and the oldest leaves cell 4.

This mechanism is central to saving memory bandwidth. Computed directly, each sample would be read five times, since it belongs to five consecutive output windows. With the sliding window, **each sample is read exactly once** and then slides through all five positions, cutting memory accesses to one fifth. The system has eight sliding windows, enough for Conv4, the layer with most input channels. Table 3.10 lists the port interface.

Table 3.10. Mapping of sliding-window cells to multiplication branches.

| Multiplication branch | Physical cell read | Corresponding sample | Paired with weight |
|---|---|---|---|
| k = 0 | cell 4 (oldest) | x[t − 2] | w[0] |
| k = 1 | cell 3 | x[t − 1] | w[1] |
| k = 2 | cell 2 (centre) | x[t] | w[2] |
| k = 3 | cell 1 | x[t + 1] | w[3] |
| k = 4 | cell 0 (newest) | x[t + 2] | w[4] |

**Zero padding at both ends of the sequence.** A convolution with padding 2 needs the first two and last two positions treated as 0. The hardware uses a padding flag asserted when the read address falls outside the valid region — below 2 at the start or from in_len + 2 at the end. When asserted, the load cell of the window takes 0 instead of memory data. Table 3.11 lists the cases.

Table 3.11. Conditions generating zero padding.

| Region | Address condition | Data | Reason |
|---|---|---|---|
| Start | address < 2 | 0 | Padding = 2 |
| Valid | 2 ≤ address < in_len + 2 | data from memory | data of the sequence |
| End | address ≥ in_len + 2 | 0 | Padding = 2 |

The padding flag must be **passed through a register stage** before use, because on-chip FPGA memory has a synchronous read latency of one cycle: an address issued now returns data next cycle. Used at once, the flag would arrive one cycle ahead of the data and the zeros would land in the wrong places. Padding at the end is equally necessary, or the engine would read cells outside the valid region and feed spurious values into the computation.

**The synchronization delay chain.** This is the most delicate timing detail of the engine. The pipeline from branch selector to accumulator register is exactly five cycles deep.

Table 3.12. Five-cycle pipeline depth, selector to accumulator register.

| Cycle | Event |
|---|---|
| N | *a* sliding-window selection → *mux_comb* (combinational read); weight read address issued |
| N+1 | *mux_s1* ← *mux_comb*; *w_packed* ← weights (synchronous read) |
| N+2 | *prod0..prod4* ← *mux_s1* × *w_packed* (S1 multiplication) |
| N+3 | *sum01*, *sum23* (S2) |
| N+4 | *sum0123* (S3) |
| N+5 | *tree_out* (S4) — accumulator update edge reads *a_d5*/*inch_d5*/*ce_d5* |

Hence the accompanying control signals cannot be used directly at the end of the pipeline: at cycle N+5, when the accumulator updates, the counter *a* at the head of the pipeline has already moved on. Three signals — the channel counter *a*, the number of input channels *in_ch* of the layer in progress, and the compute-enable signal *compute_en* — are therefore each delayed by five cycles through three five-stage shift-register chains, one cycle per stage, named after the original signal plus the suffix *_d* together with the delay stage number:

– *a_d1* → *a_d2* → ... → *a_d5*: the one- to five-cycle delayed channel counter. The final-stage value *a_d5* tells the accumulator which input channel the arriving sum belongs to, that is, whether it is the first and the accumulator must be initialized.

– *inch_d1* → ... → *inch_d5*: the delayed versions of the input-channel count *in_ch* — telling the accumulator when all channels of the layer are summed, result can be latched and rescaled.

– *ce_d1* → ... → *ce_d5*: the delayed versions of the compute-enable signal *compute_en* — allowing the accumulator to be updated only with valid data, discarding the spurious values of the sliding-window priming phase.

All three are pure shift registers: each cycle the value of one stage passes to the next *a_d5* ← *a_d4* ← ... ← *a_d1* ← *a*. Only the three signals at the final stage, *a_d5*, *inch_d5* and *ce_d5* , are wired to the ports of the eight convolution-pool units; intermediate stages exist only for the delay. All three chains are cleared to 0 by the window clear pulse *srw_rst* at a layer transition, so the new layer starts clean.

This depth of five cycles must match the real pipeline depth exactly. An error of one cycle would attribute each convolution sum to the wrong input channel, shift the max-pooling window and make every result from that layer on diverge from the reference model

**Weight storage:** the weights are preloaded into four ROMs. The store is indexed by the layer in progress and the channel counter, delivering the 40-bit weight word one cycle before the multipliers need it, in step with the window. It has a four-input combinational selector on the layer index, a selector on the input channel, and a register stage. As the weights are embedded, it has no bus write port and the configuration is fixed after synthesis.

### 3.2.3. Design of the gap-fc-argmax unit

The **gap-fc-argmax unit** block performs the last three network steps: global average pooling, the FC layer and the search for the largest logit. It runs as a sequential sub-state machine taking **22 cycles** from the end of Conv4. It comprises three sub-blocks: gap_unit, fc_unit and argmax_unit. Figure 3.8 gives the block diagram.

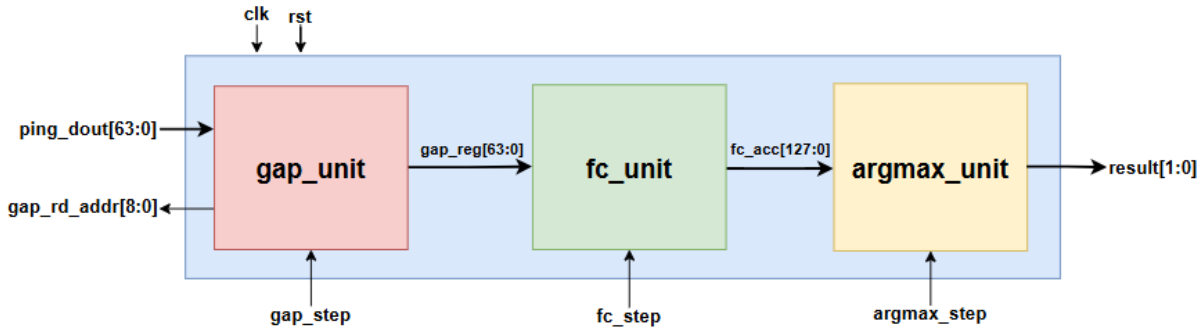



Figure 3.8. The gap-fc-argmax unit.

Table 3.13. Cycle budget of the gap-fc-argmax unit.

| Stage | Cycles | Operation |
|---|---|---|
| GAP | 6 | Read and sum 4 positions of 8 channels, then take the mean |
| FC | 10 | Multiply and accumulate 8 inputs for 4 output channels |
| FC flush | 1 | Add the remaining product still in the pipeline |
| Argmax | 4 | Sequentially compare the 4 logits |
| Done | 1 | Latch the result and assert the completion signal |
| Total | 22 | |

Table 3.14. Port interface of the gap-fc-argmax unit.

| Port | Direction | Width | Description |
|---|---|---|---|
| clk, rst | in | 1 | Clock and reset |
| fc_sub_state | in | 3 | Sub-state: GAP / FC / flush / Argmax / Done |
| gap_step | in | 4 | Step counter of the GAP stage, 0..5 |
| fc_step | in | 4 | Step counter of the FC stage, 0..9 |
| argmax_step | in | 2 | Step counter of the Argmax stage, 0..3 |
| ping_dout | in | 64 | Conv4 output read from the Ping bank, 8 channels packed |
| gap_rd_addr | out | 9 | Read address sent to the Ping-Pong RAM |
| result | out | 2 | Index of the class with the largest logit |

**Global average pooling (GAP):** For each of the eight channels the block sums the four remaining Conv4 outputs and takes their mean. As the input has already passed ReLU, the averaging division is only **a right shift by two bits**, which is equivalent to division by 4 with the floor taken.

**The fully connected layer (FC):** The eight values from global average pooling are multiplied by the 4 × 8 weight matrix to give four logits. The block uses **four parallel multipliers**, one per output channel, and scans the eight inputs through a two-stage pipeline: one cycle latches the input, the next multiplies, the next adds into the accumulator. The FC bias is **preloaded into the accumulator** at the very first step, so it is added free of charge during accumulation with no separate addition. As the pipeline has two cycles of latency, the last product is still inside it when the FC stage ends, so one flush cycle is needed to add it — hence the separate row for this step in the cycle-budget table.

**Maximum search (Argmax):** The four logits are compared over four cycles using one register for the largest value and one for its index. On the first cycle logit 0 is loaded; on the next three, a logit replaces the value held only if strictly greater. The result is the predicted class index, presented at the result port.

### 3.2.4. Design of the controller and the auxiliary blocks

The controller is the component that coordinates the entire operation of the design. It is implemented as **a single unified finite state machine**that controls in sequence both the engine for the four convolutional layers and the output classifier, rather than being split

into separate controllers. All timing is then in one place, making the total cycle count far easier to reason about and verify than control logic scattered across blocks. Figure 3.9 gives the state diagram.

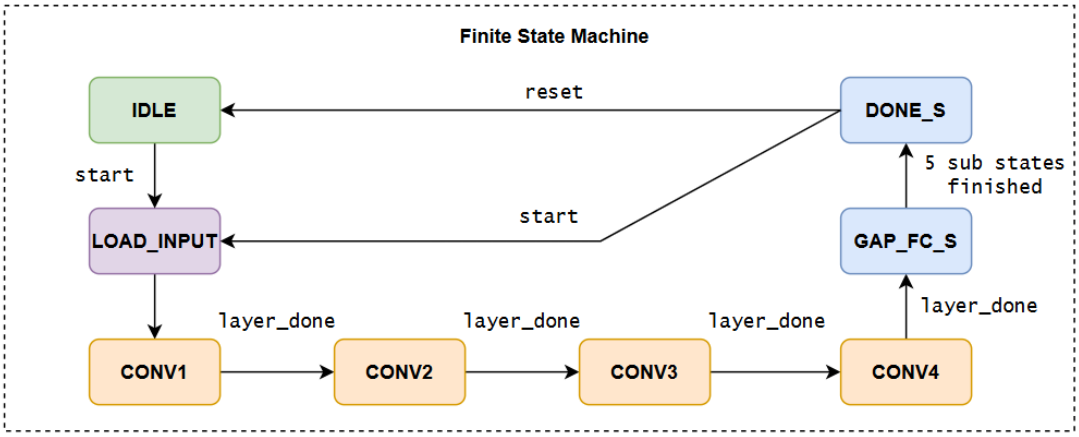



Figure 3.9. State diagram of the controller.

The state machine comprises eight main states encoded in 3 bits, in Table 3.15.

Table 3.15. States of the controller.

| State | Code | Role |
|---|---|---|
| IDLE | 0 | Awaiting a command; the busy signal is 0 |
| LOAD_INPUT | 1 | Load the Conv1 parameters, clear the window and pooling counters |
| CONV1 | 2 | Convolution of layer 1, reading data from the Input RAM |
| CONV2 | 3 | Convolution of layer 2 |
| CONV3 | 4 | Convolution of layer 3 |
| CONV4 | 5 | Convolution of layer 4, with ReLU enabled |
| GAP_FC_S | 6 | Global average pooling, fully connected and maximum search |
| DONE_S | 7 | Latch the result and issue the done pulse |

**Two nested counters.** In the convolution states, the controller maintains two counters. The input-channel counter $a$ runs from $0$ to $in\_ch - 1$ and wraps around, while the output-position counter $t$ counts positions along the sequence. Each time $a$ wraps around, that is when *shift_en* is asserted, $t$ increments by one and the sliding window shifts one cycle. This nesting makes the engine spend exactly $in\_ch$ cycles on each output position, one input channel per cycle, matching the multi-channel accumulation of Section 3.2.1.

**Per-layer parameters.** The controller issues all parameters of the layer in progress — input channels, input and output lengths, rescaling shift bits, ReLU enable and output-channel mask — and reloads them at each transition. In this single-load design they are fixed constants, as look-up functions of the layer index. Table 3.16 gives the values.

Table 3.16. Control parameters for each layer.

| Layer | in_ch | in_len | out_len | nb | relu_en | cp_en | Output channels used |
|---|---|---|---|---|---|---|---|
| Conv1 | 1 | 2500 | 500 | 8 | 0 | 0x0F | 4 |
| Conv2 | 4 | 500 | 100 | 7 | 0 | 0x0F | 4 |
| Conv3 | 4 | 100 | 20 | 6 | 0 | 0xFF | 8 |
| Conv4 | 8 | 20 | 4 | 7 | 1 | 0xFF | 8 |

**Layer completion and bank swapping.** The controller detects the end of a layer when the write address reaches its last value while the write signal is asserted, that is when the last pooled value has been written. On that cycle it does four things: loads the next layer's parameters; resets the channel, position, write-address and priming counters; clears the sliding window and pooling counter; and **swaps the roles of the two memory banks** by toggling the bank-select signal, so the bank just written becomes the read bank of the next layer. When Conv4 ends, however, the state machine goes to the output classification state and starts its first sub-state instead.

**Coordination of the output classification by five sub-states.** GAP_FC_S is not monolithic: it contains a five-step sub-state machine, each step running a prescribed number of cycles, totalling the 22 cycles of Table 3.13. Table 3.17 lists the sub-states.

Table 3.17. Five sub-states of the output classification stage.

| Sub-state | Cycles | Controlling counter | Transition condition |
|---|---|---|---|
| GAP_SUB | 6 | gap_step : 0 → 5 | gap_step = 5 → FC_SUB |
| FC_SUB | 10 | fc_step : 0 → 9 | fc_step = 9 → FC_FLUSH_S |
| FC_FLUSH_S | 1 | | Immediate transition → ARGMAX_SUB |
| ARGMAX_SUB | 4 | argmax_step: 0 → 3 | argmax_step = 3 → DONE_SUB |
| DONE_SUB | 1 | — | Latch result, assert done → DONE_S |

Putting the post-processing coordination in the common controller, rather than a separate state machine inside the classification block, leaves that block as pure datapath. All timing in one place eases verification and the total cycle count.

Furthermore, in the DONE_S state the controller preserves the result while still **accepting a new start pulse** returning it to input loading without resetting the whole system. Consecutive ECG windows can thus be processed in continuous-stream monitoring with no initialization cycles lost between runs.

**The Ping-Pong memory.** The inter-layer buffer memory consists of **two banks**, each of 8 channels × 512 cells × 8 bit. The 512 comes from rounding the 500 actual values up to suit the M10K mode on the Cyclone V. The principle is double buffering: while the current layer reads the first bank it writes results to the second, and the roles swap at each transition. The read port is synchronous with one cycle of latency; the write port has one enable per channel. The output bank selector is combinational logic after the read-data register, one level of logic, so it does not affect the critical path.

**The input memory.** This memory stores one electrocardiogram window of **2500 × 8 bit**, as a simple dual-port memory: one write port for the input and one synchronous read port of one cycle latency for the engine. Data are written before the start command. Only Conv1 reads it directly; Conv2 to Conv4 read the Ping-Pong memory.

Table 3.18 summarizes the memory organization of the whole system.

Table 3.18. On-chip memory organization.

| Memory | Size | Port type | Mapping | Read by |
|---|---|---|---|---|
| Input RAM | 2500 × 8 bit | Simple dual-port | M10K | Conv1 |
| Ping-Pong RAM | 2 × 8 ch × 512 × 8 bit | Concurrent read / write | 16 × M10K | L+1; GAP |
| Weight ROM | 116 words × 40 bit | Synchronous read | Register array | engine unit |
| ROM bias | 32 × 32 bit | Synchronous read | MLAB | engine unit |
| FC weight ROM | 32 × 8 bit | Combinational read | Register array | fc_unit |

### 3.2.5. Dataflow of the CNN system

The 2,500 quantized ECG samples go into the input memory and the start pulse is issued; the state machine moves from waiting to input loading and then Conv1. There the engine reads the input memory through the sliding window, computes four output channels, applies max-pooling and writes 500 values per channel to one Ping-Pong bank; with one input channel, each output position takes one cycle. The next three layers read the previous feature map from the Ping bank, accumulate over several input channels, rescale, pool and write to the other bank, the sequence shortening as they go. After Conv4 the classifier averages the eight channels, computes four logits and finds the largest. The state machine then asserts the completion pulse and latches the result for the host

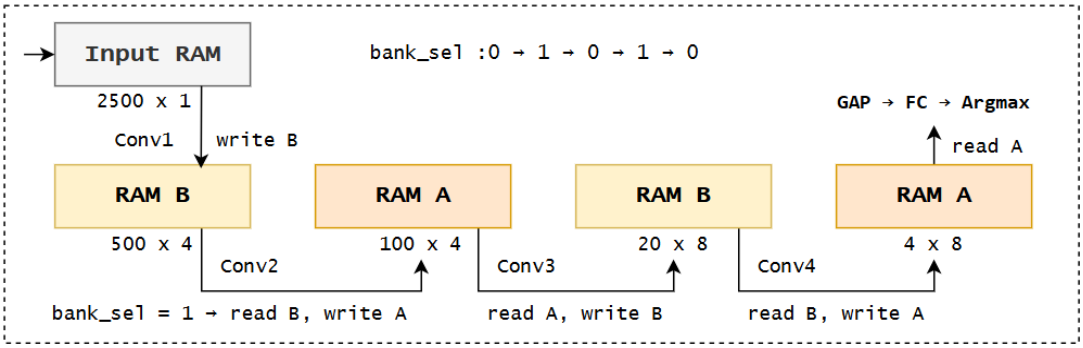



Figure 3.10. Dataflow through the four convolutional layers and the bank swap.

An important characteristic of this dataflow is that it is entirely **deterministic** : the controller is a state machine with a predetermined number of steps and no data-dependent branch, so every input takes exactly the same number of cycles. This is a strong advantage for real-time monitoring, since response time is guaranteed strictly, not merely on average. Table 3.19 analyses the cycle count of one inference.

Table 3.19. Cycle-count analysis of a single inference.

| Stage | Derivation | Cycles |
|---|---|---|
| Conv1 | 1 input channel × 2,500 positions | 2,500 |
| Conv2 | 4 input channels × 500 positions | 2,000 |
| Conv3 | 4 input channels × 100 positions | 400 |
| Conv4 | 8 input channels × 20 positions | 160 |
| Sliding-window priming | 5 shift cycles × input channels, all 4 layers | 85 |
| GAP / FC / Argmax | fixed, per Table 3.13 | 22 |
| Pipeline drain, state trans | pipeline latency at each layer transition | 49 |
| Total | | 5,216 |

## 3.3. INTERFACE AND SYSTEM CONTROL INTEGRATION

The computational core is bus-independent and cannot talk to a computer by itself. Two layers are needed to run on the board: a wrapper acting as bus protocol converter, and a physical bridge between computer and FPGA.

### 3.3.1. Integration of the Avalon-Memory Mapped Wrapper with the CNN core

The top module of the design is a **thin wrapper** of only two components — the bus converter and the computational core — plus the input memory in the wrapper. The bus converter translates the Avalon-Memory Mapped protocol into the core's control signals: which address loads data into memory, which bit is the start command, and where status and result are read.

The bus converter has a 14-bit address space divided into two functional regions, as shown in Table 3.20.

Table 3.20. Avalon-MM address map of the hardware design.

| Address | Type | Name | Function |
|---|---|---|---|
| 0x0003 | Write | start | The command run and clears done flag |
| 0x0004 | Read | status | Bit 0 = busy, bit 1 = complete |
| 0x0005 | Read | result | Predicted class, 2 bits |
| 0x1000 – 0x19C3 | Write | data window | Fast loading of 2,500 samples, one byte per write word |

The host writes 2,500 ECG samples into the data window (0x1000–0x19C3), one byte of input memory per write word, then writes the start bit to 0x0003 to launch an inference; the controller leaves the wait state, runs the four convolutional layers and the classifier, and asserts done. The host polls the status register at 0x0004 until the completion bit is 1, then reads the class at 0x0005.

### 3.3.2. Integration system with the JTAG-to-Avalon IP to communicate with a PC

The design is deployed on the board using **a JTAG-to-Avalon bridge**.

The on-board system is assembled in the Quartus Platform Designer tool. It has three components: the JTAG-to-Avalon IP core as master; the clock and reset bridges; and the hardware as slave, its Avalon-MM interface wired directly to the master. Figure 3.11 shows the interconnection.

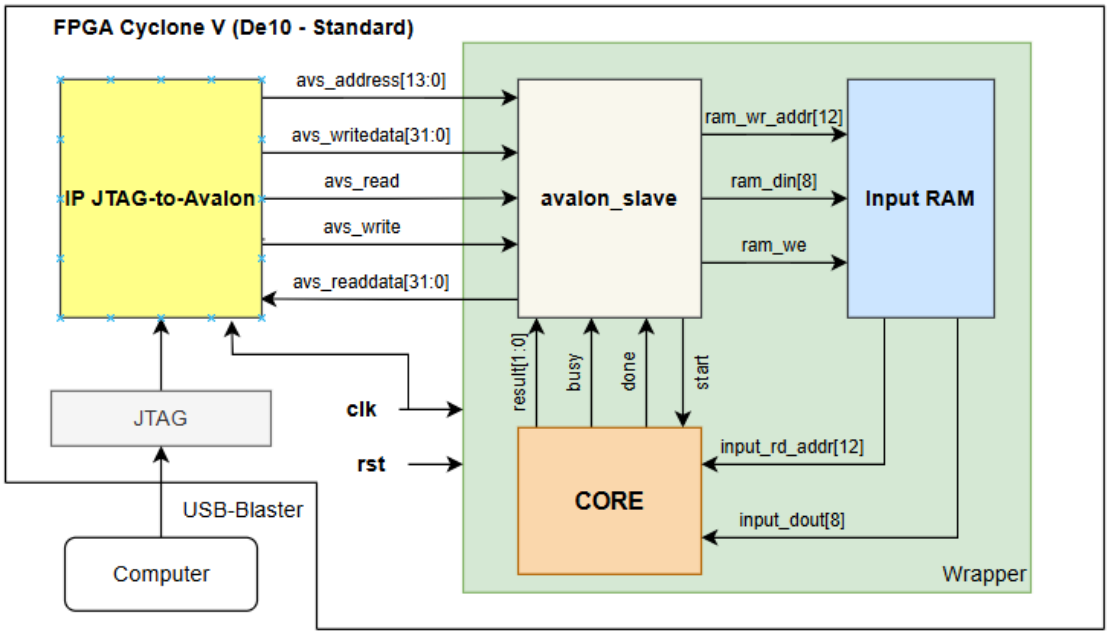



Figure 3.11. On-board integration with the JTAG-to-Avalon bridge.

On the computer, the System Console tool supplied with Quartus runs a control script written in Tcl and acts as the host. The steps performed are presented in Table 3.21.

Table 3.21. System control steps performed by a script in the System Console.

| Step | Operation | Details |
|---|---|---|
| 1 | Configure the device | Load the .sof file into the FPGA over USB-Blaster and check the JTAG link |
| 2 | Open the communication channel | Discover and open the JTAG-to-Avalon master to obtain a bus read/write path |
| 3 | Load the data | Write the 2,500 samples of one record into the data window at 0x1000 |
| 4 | Issue the run command | Write the start bit to address 0x0003 |
| 5 | Wait completion | Poll the status register at 0x0004 until the completion bit is 1 |
| 6 | Read the result | Read the predicted class at address 0x0005 |
| 7 | Iterate and compare | Repeat from step 3 for the entire test set and compare against the true labels |

The system is now complete, from software model through hardware core to the host interface.

# CHAPTER 4: IMPLEMENTATION RESULTS AND EVALUATION

## 4.1. EVALUATION METHODOLOGY

### 4.1.1. Model performance

Arrhythmia classification is a multi-class problem with four labels: atrial fibrillation (AFIB), supraventricular tachycardia (GSVT), sinus bradycardia (SB) and normal sinus rhythm (SR). As the classes are unbalanced, both the confusion matrix and per-class metrics are used, not overall accuracy alone.

The confusion matrix is of order four; its element in row i, column j is the number of samples of true class i predicted as class j, and the diagonal holds the correct ones. For each class, TP counts positives predicted correctly, FP samples of other classes predicted as this one, FN samples of this class predicted as another, and TN the rest. The metrics follow:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{4.2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4.3}$$

$$\text{F}_1 - \text{score} = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{4.4}$$

Accuracy is the proportion of correctly classified samples in the test set. Precision gives the percentage of samples predicted in a class that truly belong to it, reflecting false alarms. Recall gives the percentage of samples truly in the class that the model detects, reflecting misses — clinically important for pathological classes such as AFIB. The F1-score is their harmonic mean and is the main comparison metric here.

The ROC curve relates the true positive rate to the false positive rate as the decision threshold varies:

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN} \tag{4.6}$$

The area under the ROC curve, AUC, lies in [0, 1] and is the probability that the model scores a random positive above a random negative. For four classes it is computed one-versus-rest per class and macro-averaged. Interpretation matters: AUC judges the quality of the discriminative score independently of threshold, whereas F1 depends on a specific threshold (here argmax). Hence a high AUC with a markedly lower F1 points to the position of the decision boundary, not to the model's representational capacity — an observation used in Section 4.2.2.

### 4.1.2. Hardware performance

Hardware performance is assessed through a group of quantities: latency, throughput, resource utilization and power.

#### Latency and throughput

The controller is a state machine with a fixed number of steps and no data-dependent branch, so the latency of one inference is constant and follows from the cycle count. Throughput is the number of inferences per second when they run back to back:

$$T_{\text{latency}} = \frac{N_{\text{cycles}}}{f_{\text{clk}}} \tag{4.7}$$

$$\text{Throughput} = \frac{1}{T_{\text{latency}}} = \frac{f_{\text{clk}}}{N_{\text{cycles}}} \tag{4.8}$$

#### Resources, power and energy

Resource use is taken from the Quartus Prime report: ALMs (Adaptive Logic Modules, the basic programmable logic units of the Cyclone V), logic registers, M10K memory blocks and hard DSP multipliers, in absolute numbers and as a percentage of device capacity.

Power is estimated with PowerPlay and split into a static component (die leakage, independent of activity) and a dynamic one (signal switching). The energy per inference — decisive for battery-powered wearables — is:

$$E_{\text{inference}} = P_{\text{total}} \times T_{\text{latency}} \tag{4.9}$$

## 4.2. MODEL TRAINING RESULTS

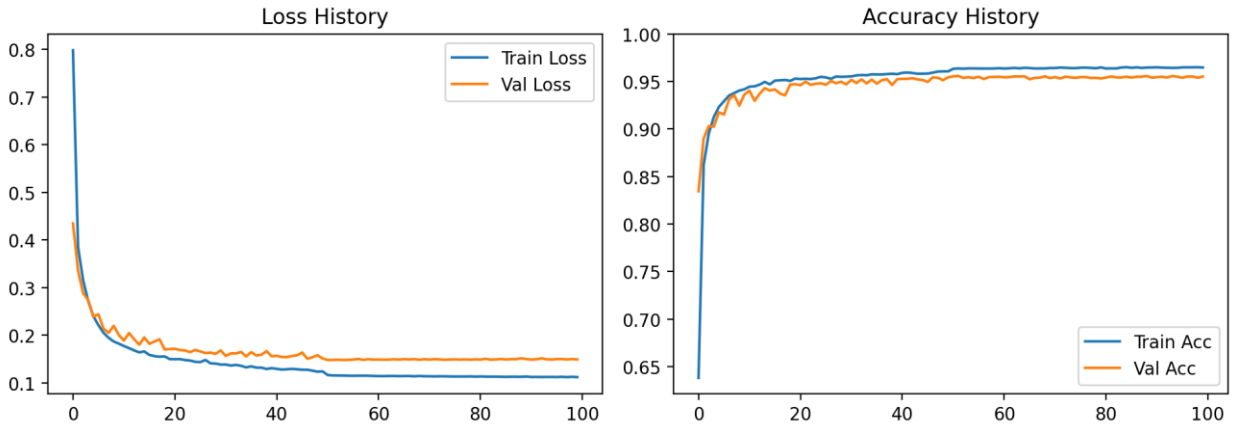



Figure 4.1. Loss (a) and accuracy (b) curves for the training and validation sets

Figure 4.1 shows the loss falling rapidly over the first 20 epochs and then flattening; validation accuracy tracks training accuracy about one percentage point behind and stays stable to the end, so the model is not overfitting.

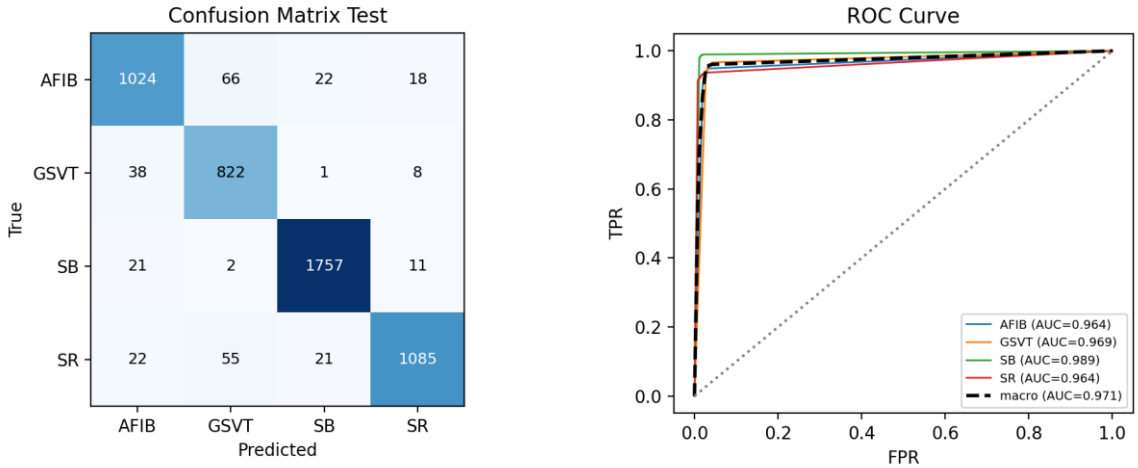
### 4.2.1. Results on the training dataset

The model was trained and evaluated on the Chapman dataset of 33,143 ECG records, split by patient 70/15/15, with a test set of 4,973 records. Table 4.1 gives the accuracy of the three model versions; the INT8 version is evaluated bit-accurately, so this is the figure the hardware produces. Table 4.2 gives the per-class detail.

Table 4.1. Accuracy at each processing step (Chapman test set, 4,973 records)

| Model version | Parameters | Accuracy (%) | F1-macro | macro-AUC |
|---|---|---|---|---|
| Full float32 | 1,244 | 95.35 | 0.9478 | — |
| Float32 after channel pruning | 640 | 95.03 | 0.9446 | 0.9938 |
| Bit-accurate | 640 | 94.27 | 0.9356 | 0.9712 |

Channel pruning cuts the parameter count by 48.55% (1,244 → 640) for only 0.32 percentage points of accuracy. Quantization from float32 to INT8 costs a further 0.76 points of accuracy and 0.90 points of macro-F1; prediction agreement between the two versions is 0.9761.




*(a) Confusion matrix* *(b) ROC curves*

Figure 4.2. Confusion matrix (a) and ROC curves (b) on the Chapman test set.

Table 4.2. Precision, recall and F1-score for each class (Chapman)

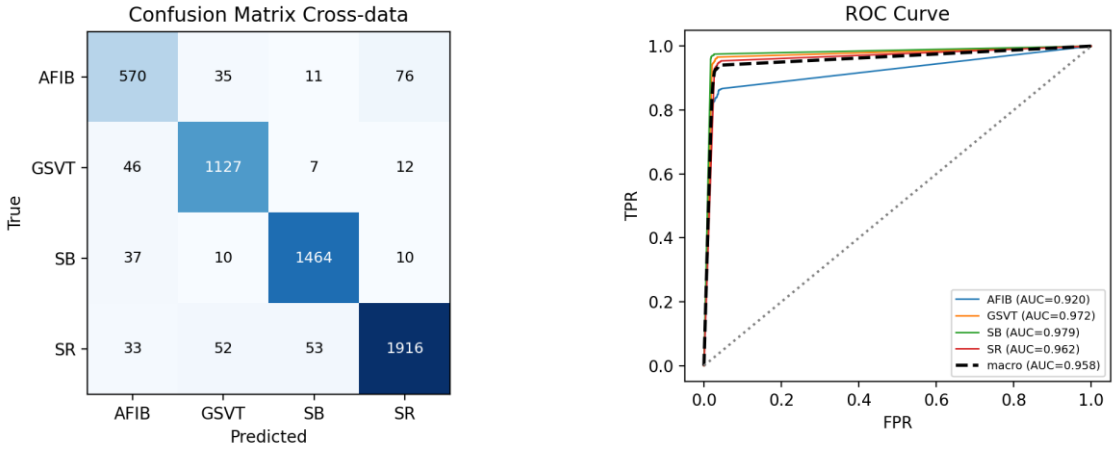
| Class | Precision | Recall | F1-score | Samples |
|---|---|---|---|---|
| AFIB (atrial fibrillation) | 0.9267 | 0.9062 | 0.9163 | 1,130 |
| GSVT (supraventricular tachycardia) | 0.8698 | 0.9459 | 0.9063 | 869 |
| SB (sinus bradycardia) | 0.9756 | 0.9810 | 0.9783 | 1,791 |
| SR (normal sinus rhythm) | 0.9670 | 0.9172 | 0.9414 | 1,183 |
| Macro average | — | — | 0.9356 | 4,973 |

The INT8 model reaches 94.27% accuracy and 0.9356 macro-F1 with only 640 parameters. SB is the best class (F1 = 0.9783), sinus bradycardia having a distinct frequency signature. The hardest are GSVT (F1 = 0.9063, precision 0.8698) and AFIB (recall 0.9062). The confusion matrix in Figure 4.2 shows two main confusions: AFIB taken for GSVT (66 samples) and SR taken for GSVT (55 samples).

The ROC curves in Figure 4.2b complement the confusion matrix: the matrix reflects one decision threshold only (the largest logit), whereas the ROC sweeps all thresholds and so measures separability independently of threshold. The macro-averaged area under the curve is 0.971, highest for SB (0.989), with AFIB and SR both at 0.964 — an ordering consistent with the matrix, showing that most remaining errors lie at the boundary between the sinus rhythm classes rather than in poor separability.

### 4.2.2. Results on the cross-dataset test set

The model trained on Chapman was applied directly to the Georgia dataset of 5,459 records with no retuning (zero-shot). Georgia was acquired with different equipment at a different institution, so this is a far-transfer test, more demanding than one within the same data family. Table 4.3 gives the per-class results.




*(a) Confusion matrix* *(b) ROC curves*

Figure 4.3. Confusion matrix (a) and ROC curves (b) on the Georgia set.

Table 4.3. Precision, recall and F1-score for each class (Georgia)

| Class | Precision | Recall | F1-score | Samples |
|---|---|---|---|---|
| AFIB (atrial fibrillation) | 0.8309 | 0.8237 | 0.8273 | 692 |
| GSVT (supraventricular tachycardia) | 0.9208 | 0.9455 | 0.9329 | 1,192 |
| SB (sinus bradycardia) | 0.9537 | 0.9625 | 0.9581 | 1,521 |
| SR (normal sinus rhythm) | 0.9513 | 0.9328 | 0.9420 | 2,054 |
| Macro average | — | — | 0.9151 | 5,459 |

Table 4.4. float32 vs INT8 on the Georgia cross-dataset set

| Metric | Float32 | INT8 |
|---|---|---|
| Accuracy | 0.9291 | 0.9300 |
| F1-macro | 0.9142 | 0.9151 |
| macro-AUC | 0.9813 | 0.9580 |
| Prediction agreement INT8 ↔ float32 | — | 0.9749 |

On the cross-dataset test set, the INT8 version attains an accuracy of 93.00% and a macro-F1 of 0.9151, only about 1.3 percentage points below the in-distribution result — a modest degradation for a far-transfer setting.

The macro-averaged area under the curve here is 0.958 (Figure 4.3b). Notably, AFIB has an AUC of 0.920, the lowest of the four but far higher than its F1-score suggests. A high AUC with a low F1 shows that the model still ranks AFIB records correctly and that only the decision threshold learned on Chapman is no longer optimal on the new distribution; the far-transfer loss thus comes chiefly from a threshold shift, not from weaker feature recognition.

## 4.3. FUNCTIONAL SIMULATION RESULTS

### 4.3.1. Functional simulation of the convolution–pool unit

This test environment drives a single convolution-pool unit directly, bypassing the delay chains of the engine unit. The testbench sets the input signals directly: the five weight coefficients W and the five data samples *x_in* of the convolution window, the scaled value *bias* , the number of shift bits *nb[3:0]*, the flag *relu_en*, the channel index *a_in* and the compute-enable flag *compute_en_in*. The observed outputs are the value *pool_out* and the pulse *pool_write*. Since the pooling stage counts a point only when both *relu_v* and *compute_en_in* are high, and since *relu_v* is delayed by 9 cycles, the testbench must load all points and keep the pipeline running so the last enters the max-pool window; the pulse *pool_write* is latched for checking. Table 4.5 presents the test plan for the block. The simulation waveforms of the block are given in Figure 4.4.

Table 4.5. Test plan for the convolution-pool unit

| Group | Test case | Verification objective |
|---|---|---|
| Basic datapath | TC01a–TC01b | Empty pipeline and pipeline with multiply–accumulate |
| Round half up | TC02a–TC02b | Rounding up at the boundary and no rounding |
| Saturation | TC03–TC06 | Upper/lower limits of ±127, including the exact boundary values |
| relu | TC07–TC09 | Negative clipping on; negatives kept when off (Conv1–3) |
| maxpool | TC10–TC13b | Maximum at every window position, and two consecutive windows |
| Multi-channel accumulation | TC14–TC15 | Accumulation over 4 and 8 channels (as in Conv4) |
| Control and reset | TC16–TC19 | Idle cycles, pooling counter reset, global RST |

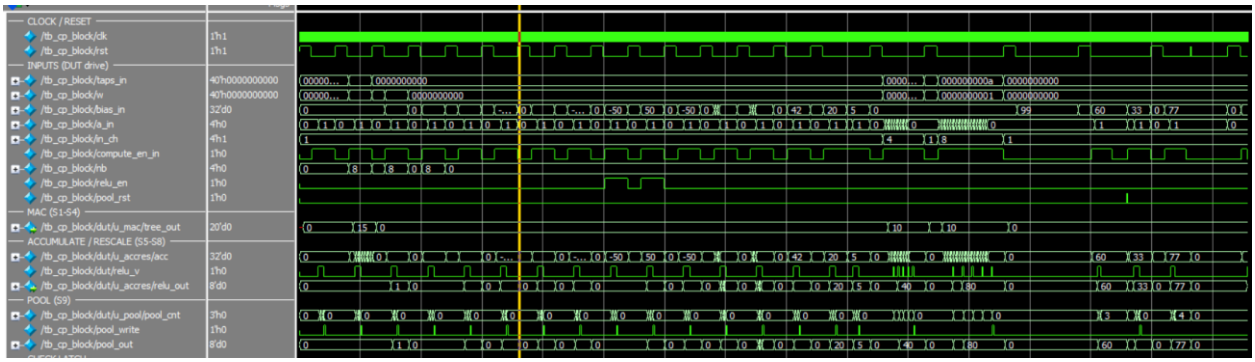


Figure 4.4. Waveforms of the convolution-pool unit.

Result: all 23 cases pass. Notable are TC02a — with bias = 128 and nb = 8 the correct result is 1 under round-half-up, whereas truncation gives 0 — and TC15, confirming the accumulation path in the 8-channel configuration, the heaviest case in the network.

### 4.3.2. Functional simulation of layer-by-layer operation

Here the engine unit, controller and ping-pong memory are integrated to verify control and timing while running Conv1 and moving on to Conv2 and Conv3. The input is synthetic; what matters is not the values but the timing, addresses and write-enable conditions. The signals observed are: *layer_state* of the state machine, the bitmask *cp_en* identifying which channels are active, the flag *bank_sel* selecting the memory bank, the pulse *pool_write*, the write-enable signal *pong_we* and the write address *pong_addr*. Table 4.6 summarizes the results and Figure 4.5 shows the layer-transition waveforms.

Table 4.6. Test plan for layer operation

| TC | Item tested | Expected result |
|---|---|---|
| TC01 | No write pulse during the 14 preloading cycles | Inhibition works while the pipeline is not yet full |
| TC02 | Write address at the last pulse of Conv1 | pong_addr = 499 |
| TC03 | Total number of pool_write pulses of Conv1 | 500 pulses (2500 divided by 5) |
| TC04 | State machine transition | layer_state advances to CONV2 |
| TC05 | Bank swap after a layer completes | BANK_SEL changes 0 → 1 |
| TC06 | Channel masks of Conv1 and Conv2 | cp_en = 0x0F (4 channels) |
| TC07 | Write inhibition on unused channels | PONG_WE[4..7] inhibited |
| TC08 | Channel masks of Conv3 and Conv4 | cp_en = 0xFF (8 channels) |

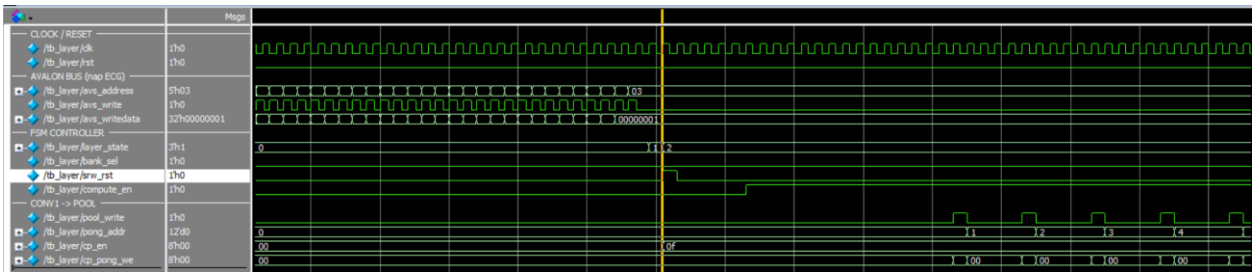


Figure 4.5. Waveforms of the transition between convolutional layers.

Result: all 8 cases pass. TC03 confirms that each layer produces the designed number of outputs, and TC01 that write inhibition during preloading works — a timing fault that had caused a one-cycle misalignment between convolution window and accumulator during development.

### 4.3.3. Simulation of test-set samples

This environment exercises the whole system at top level with a real ECG sample from the Chapman test set. The 2,500 INT8 values of the sample are written one by one into the input memory over the Avalon-MM bus using *avs_address*, *avs_writedata* and *avs_write*; the start bit is then written to the control register and the state machine leaves IDLE, passing through LOAD_INPUT, CONV1, CONV2, CONV3, CONV4 and GAP_FC. Weights and biases are loaded into the ROMs by $readmemh at simulation start. Each convolutional layer reads one ping-pong bank and writes the other, the flag *bank_sel* toggling after each layer. On completion GAP/FC/argmax gives the class index *result[1:0]* and done is latched; the result is read via *avs_readdata*. Table 4.7 presents the comparison results. Figure 4.6 is a capture of the system in actual operation.

Table 4.7. Bit-accuracy test plan using test-set samples

| No. | Checkpoint | Elements compared | Significance |
|---|---|---|---|
| 1 | input_int8 | 2,500 | Data enter the memory correctly over the bus |
| 2 | after_pool1 | 2,000 | Conv1 + pooling (500 × 4 channels) |
| 3 | after_pool2 | 400 | Conv2 + pooling (100 × 4 channels) |
| 4 | after_pool3 | 160 | Conv3 + pooling (20 × 8 channels) |
| 5 | after_pool4 | 32 | Conv4 + ReLU + pooling (4 × 8 channels) |
| 6 | after_gap | 8 | Floor integer division of the GAP stage |
| 7 | logits_fc | 4 | FC layer logits before argmax |

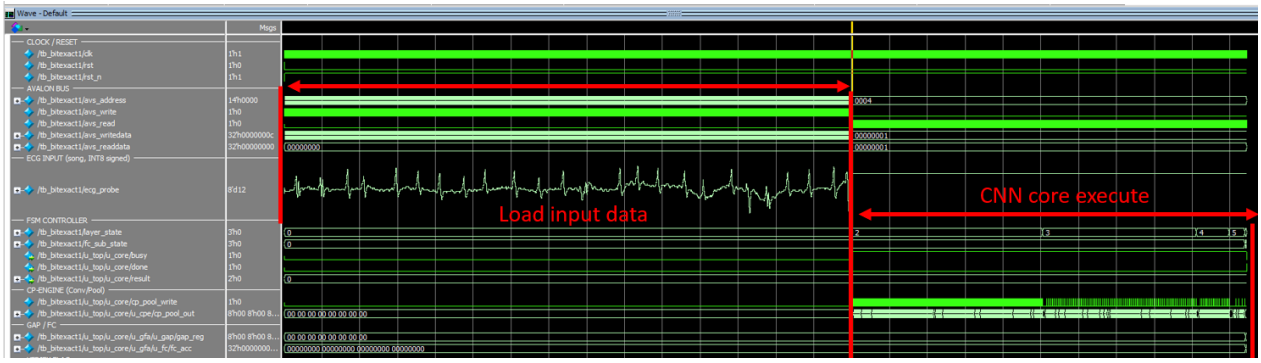



Figure 4.6. Waveforms of one complete bit-accurate inference.

Result: 7 of 7 checkpoints are bit-accurate, with a maximum absolute deviation of 0. The hardware label matches the reference label. As the deviation is zero at every stage, not only at the output, the circuit clearly implements the quantization specification exactly, with no mutually cancelling errors.

### 4.3.4. Simulation of cross-dataset samples

This environment matches Section 4.3.3 but uses an ECG sample from the Georgia cross-dataset test set, the weight ROMs still holding the Chapman coefficients. Figure 4.7 shows the results on the computer.

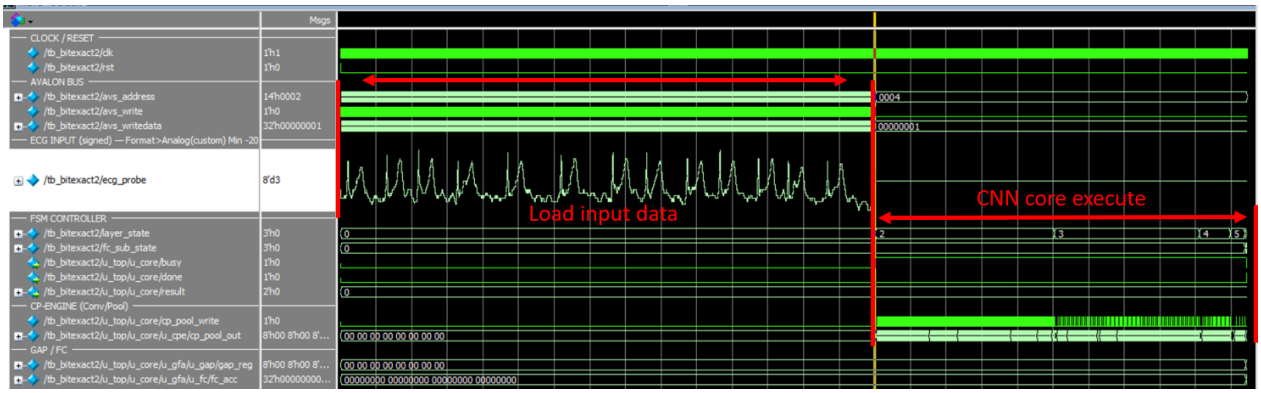



Figure 4.7. Waveforms of a Georgia cross-dataset sample.

Result: the circuit label agrees with the Python reference, consistent with the INT8–float32 agreement of Table 4.4.

## 4.4. EXPERIMENTAL RESULTS ON THE FPGA BOARD

The design was programmed onto the DE10-Standard kit (Cyclone V 5CSXFC6D6F31C6), the computer communicating with it through the JTAG-to-Avalon

bridge and the System Console: a Tcl script writes the ECG data into the input memory, triggers inference and reads the result, with no operating system on the board. Figure 4.8 gives the results over the whole test set.

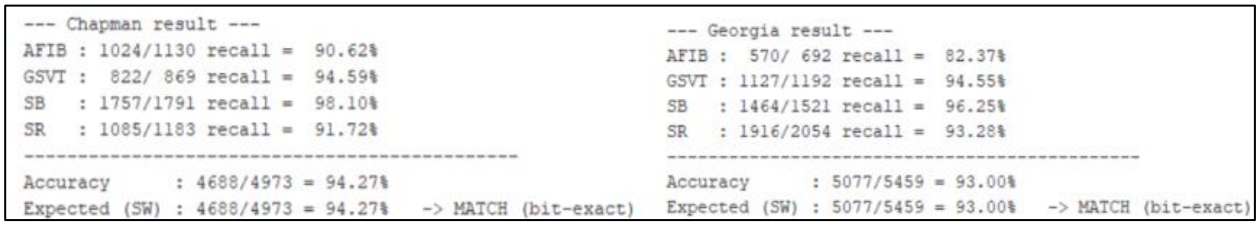



Figure 4.8. The full test set on the board, shown in the System Console.

### 4.4.1. Results for a single sample

One ECG sample is written into the input memory and the start bit asserted; the System Console reads back the done flag and result[1:0]. Table 4.8 records one sample from the principal dataset and one from the cross-dataset set.

Table 4.8. Single-sample classification on the board

| Sample | Data source | Label | result[1:0] read back | Match |
|---|---|---|---|---|
| 1 | Principal test set | GSVT | 01 | Yes |
| 2 | Cross-dataset test set | SB | 10 | Yes |

### 4.4.2. Results for the complete test set

The script loops over the samples, loading data, asserting start, waiting for done and comparing labels, then prints the aggregate accuracy. Table 4.9 summarizes.

Table 4.9. On-board vs software accuracy

| Dataset | Samples | Correct samples | On-board accuracy (%) | Software accuracy (%) |
|---|---|---|---|---|
| Principal test set | 4973 | 4688 | 94.27% | 94.27% |
| Cross-dataset test set | 5459 | 5077 | 93.00% | 93.00% |

## 4.5. RESOURCE AND PERFORMANCE EVALUATION

### 4.5.1. Resource results

The design was synthesized with Quartus Prime 25.1std Lite for the Cyclone V 5CSXFC6D6F31C6. Table 4.10 gives the resources per block, from the tool's hierarchical allocation report. Table 4.11 gives the detail per block.

Table 4.10. Resources per design block

| Block | ALM | Combinational ALUTs | Registers | Memory bits | M10K | DSP |
|---|---|---|---|---|---|---|
| avalon_slave | 10.5 | 8 | 23 | 0 | 0 | 0 |
| cnn_controller | 71.8 | 122 | 83 | 0 | 0 | 0 |
| Engine unit | 1,691.9 | 2,281 | 2,418 | 0 | 0 | 24 |
| ROM weight | 230.7 | 309 | 147 | 0 | 0 | 0 |
| GAP-FC-argmax unit | 342.2 | 567 | 374 | 0 | 0 | 4 |
| Ping-pong RAM | 25.1 | 80 | 2 | 65,536 | 16 | 0 |
| Input RAM | 0.0 | 0 | 0 | 20,000 | 4 | 0 |
| Whole design | 2,148 | 3,069 | 2,902 | 85,536 | 20 | 28 |

Table 4.11. Resource use relative to the Cyclone V 5CSXFC6D6F31C6 capacity

| Resource type | Used | Device capacity | Proportion (%) |
|---|---|---|---|
| ALM | 2,148 | 41,910 | 5 |
| Logic registers | 2,902 | 83820 | 3 |
| DSP blocks | 28 | 112 | 25 |
| M10K memory blocks | 20 | 553 | 4 |
| Block memory bits | 85,536 | 5,662,720 | 2 |
| Signal pins | 83 | 499 | 17 |

These two tables cover the hardware design alone, the part presented in Chapter 3. The bitstream programmed onto the kit in Section 4.4 also contains the data-loading infrastructure: the tool-generated JTAG-to-Avalon bridge and the reset controller.

| | |
|---|---|
| Quartus Prime Version | 25.1std.0 Build 1129 10/21/2025 SC Lite Edition |
| Revision Name | ecg_accelerator_top |
| Top-level Entity Name | ecg_accelerator_top |
| Family | Cyclone V |
| Device | 5CSXFC6D6F31C6 |
| Timing Models | Final |
| Logic utilization (in ALMs) | 2,148 / 41,910 ( 5 % ) |
| Total registers | 2902 |
| Total pins | 83 / 499 ( 17 % ) |
| Total virtual pins | 0 |
| Total block memory bits | 85,536 / 5,662,720 ( 2 % ) |
| Total DSP Blocks | 28 / 112 ( 25 % ) |

| | |
|---|---|
| Quartus Prime Version | 25.1std.0 Build 1129 10/21/2025 SC Lite Edition |
| Revision Name | jtag_top |
| Top-level Entity Name | jtag_top |
| Family | Cyclone V |
| Device | 5CSXFC6D6F31C6 |
| Timing Models | Final |
| Logic utilization (in ALMs) | 2,219 / 41,910 ( 5 % ) |
| Total registers | 3519 |
| Total pins | 2 / 499 ( < 1 % ) |
| Total virtual pins | 0 |
| Total block memory bits | 86,048 / 5,662,720 ( 2 % ) |
| Total DSP Blocks | 28 / 112 ( 25 % ) |

Figure 4.9. Resources of the hardware design and of the board bitstream

### 4.5.2. Performance results

**Operating frequency**

Table 4.12 gives the maximum frequency reported by the tool at the slowest corner (1100 mV, 85 °C).

Table 4.12. Operating frequency after synthesis

| Quantity | Value |
|---|---|
| Reported Fmax (slow model, 1100 mV, 85 °C) | 108.86 MHz |
| Timing slack at the constrained frequency | +0.814 ns |

**Latency and throughput**

The controller is a state machine with a fixed number of steps and no data-dependent branch, so the cycle count of one inference is constant and predictable per layer. As the eight output channels run in parallel, each convolutional layer takes as many cycles as input points times input channels:

$$N_{\text{layer}} = L_{\text{in}} \times C_{\text{in}} \tag{4.10}$$

The actual cycle count was measured in simulation by counting the cycles the state machine spends in each layer over one inference. Table 4.13 sets these beside the theoretical counts to expose each layer's overhead.

Table 4.13. Cycle-count analysis for each layer

| Layer | Input length | Input channels | Theoretical (length × channels) | Measured | Difference |
|---|---|---|---|---|---|
| Conv1 + pooling | 2,500 | 1 | 2,500 | 2,517 | +17 |
| Conv2 + pooling | 500 | 4 | 2,000 | 2,031 | +31 |
| Conv3 + pooling | 100 | 4 | 400 | 431 | +31 |
| Conv4 + pooling | 20 | 8 | 160 | 211 | +51 |
| GAP / FC / argmax | 4 | 8 | 22 | 22 | 0 |
| State transitions and status reads | — | — | — | 4 | +4 |
| Total | — | — | 5,082 | 5,216 | +134 |

The measurements match theory for the main computation, the difference being exactly the pipeline priming overhead of each layer. Before a layer yields its first result the sliding window must be loaded with five values, each shift costing as many cycles as there are input channels; to this are added the eleven fixed cycles of pipeline depth and max-

pooling tail. The overhead is therefore 5 × (input channels) + 11, giving 16, 31, 31 and 51 cycles for Conv1 to Conv4. Conv2 to Conv4 match exactly; Conv1 measures 17 cycles, one more than the formula, because it also bears the transition from input loading to computation. The four remaining cycles are the state transitions at either end of the inference and the latency of reading the status register over the bus.

Latency and throughput follow from the measured cycle counts by (4.7) and (4.8). Table 4.14 summarizes.

Table 4.14. Latency, throughput and energy per inference

| Quantity | 100 MHz | 108.86 MHz |
|---|---|---|
| Cycles per inference | 5,216 | 5,216 |
| Latency (µs) | 52.16 | 47.91 |
| Throughput (inferences per second) | 19,172 | 20,870 |
| Total power (mW) | 536.08 | — |
| — dynamic component (mW) | 110.89 | — |
| — static component (mW) | 412.12 | — |
| Energy per inference (µJ) | 27.96 | — |

## 4.6. COMPARISON WITH RELATED WORK

Table 4.15 compares this design with three FPGA accelerators for ECG classification, all figures taken from the respective full texts. Liu et al. (2023) is the closest comparison: same Chapman dataset with four rhythm groups, same record-level granularity and same Cyclone V family. Pham et al. (2025) represents a flexible processing-element array on a large Zynq UltraScale+ device, and Tran et al. (2026) low-dimensional hypervector computing without DSP blocks. The latter two classify individual beats on MIT-BIH, so their accuracy is not on an equal footing with the Chapman works: beat-level classification labels each beat, record-level labels a whole ten-second segment, and the different unit means a different difficulty.

The clearest advantage of this design is area: 2,148 ALMs, 5% of the device, against 51% of ALMs and 86% of registers in Liu et al. on the same board family. The difference is architectural — eight processing units replicated along the channel dimension and reused across layers, whereas Liu maps the whole network onto hardware, trading area for latency.

Table 4.15. Comparison with related work on ECG-based arrhythmia classification

| **Criterion** | **This work** | **Liu et al.** [16] | **Pham et al.** [17] | **Tran et al.** [23] |
|---|---|---|---|---|
| Target board | Cyclone V 5CSXFC6D6F 31 | Cyclone V 5CSEBA6U23I7 | Zynq UltraScale+ ZCU102 | Zynq-7000 Pynq-Z2 |
| Architecture | **1D-CNN** | 1D-CNN | 1D-CNN (Mini-InceptionNet) | LDC (hypervector) |
| Parameters | **640** | - | 6,457 | - |
| Quantization | **Int 8** | Int 8 | Fixed 16 bit | 1-bit binary |
| Dataset | **Chapman** | Chapman | MIT-BIH | MIT-BIH |
| Classification | **Rhythm (10s)** | Rhythm (10s) | Beat | Beat |
| Accuracy | **94.27 %** | 92.95 % | 99.37 % | 97.18 % |
| ALM / LUT | **2,148 ALM (5%)** | 51% | 24,685 LUT | 8,255 LUT |
| Registers | **2,902 (3%)** | 86% | 11,180 | 8,296 |
| DSP blocks | **28 DSP (25%)** | 39% | 40 DSP | 0 DSP |
| Frequency | **100 MHz** | 50 MHz | 250 MHz | 50 MHz |
| Latency | **52.16 µs** | 66 µs | 34.45 µs | 23.9 µs (core) 4,195 µs (end-to-end) |
| Throughput (inferences/s) | **19,172 (inference/s)** | 15,150 (inference/s) | 29,028 (inference/s) | - |
| Power | **536.08 mW** | 66 mW | 0.827 W | 1.709 W |

# CHAPTER 5: CONCLUSION

## 5.1. ACHIEVED RESULTS

The four groups of objectives of Section 1.1 have been met, covering the whole path from software model to a circuit running and measured on an FPGA kit.

**Regarding the model:**

– A 1D-CNN of four convolutional layers was trained to classify four rhythm groups directly from a single-lead ECG sequence.

– Channel pruning cut the parameters from 1,244 to 640 (48.55%) for only 0.32 points of accuracy, so all weights fit in on-chip memory.

– The 8-bit integer model reaches 94.27% and 0.9356 macro-F1 on the Chapman test set; zero-shot on Georgia it holds 93.00% and 0.9151.

**Regarding quantization:**

– Power-of-two scaling combined with round-half-up reduces the rescaling step to a constant addition and a bit shift, using no hard multipliers.

– A reference model reproduces the circuit's exact integer sequence as the golden reference for hardware verification.

**Regarding hardware design and verification:**

– Verilog description of a design with eight parallel convolution–pool units, the classifier, the controller and the on-chip ping-pong memory.

– Verification passed at all three levels: 23 block-level cases, 8 layer-operation cases and 7 of 7 system-level bit-accuracy checkpoints with a maximum deviation of 0 — enough to assert exact computation.

**Regarding FPGA board deployment:**

– The design uses 2,148 ALMs (5%), 28 DSP blocks (25%) and 20 M10K blocks (4%) on the Cyclone V 5CSXFC6D6F31C6, at up to 108.86 MHz.

– One inference takes a fixed 5,216 cycles: 52.16 µs latency, about 19,172 inferences per second at 100 MHz, and an estimated 536.08 mW, or 27.96 µJ per inference.

– On the DE10-Standard kit the bitstream gives 94.27% on the principal test set (4,688/4,973) and 93.00% on the cross-dataset set (5,077/5,459), matching the software.

## 5.2. LIMITATIONS

Several limitations must also be stated in order to situate these conclusions correctly.

First, the power and energy figures are not yet measurements. The 536.08 mW was estimated with PowerPlay from a switching-activity file extracted from simulation, and the tool itself rates the confidence as low because that file does not cover all combinational nodes. The figure therefore serves only for relative comparison and is not a published power measurement. The static part moreover reaches 77% of the total — a trait of the large Cyclone V device rather than of the design — which makes the energy per inference unfavourable against designs on smaller devices.

Second, the scope of the problem is narrow. The model uses one lead and classifies four rhythm groups over a whole ten-second record, whereas clinical practice needs more disorders distinguished, many recognizable only from several leads at once. Record-level classification also prevents the design from localizing an abnormality within the segment.

Third, the hardware configuration is fixed at synthesis. The weights sit in on-chip ROM, so any change of model requires resynthesis and reprogramming; weights cannot be reloaded nor the network reconfigured at run time.

Fourth, data delivery is still experimental. Data come from the host over the JTAG-to-Avalon bridge, so the hardware is not yet coupled to a real acquisition circuit and does not work on a continuous real-time stream. The evaluation therefore rests on public, preprocessed data, not on directly acquired signals with their far greater noise and baseline wander.

## 5.3. FUTURE WORK

From these limitations, four directions are proposed, from completing the present measurements to widening the scope of the system.

**Completing the power measurements:**

– Measuring the device supply-rail current during continuous inference and comparing it with the estimate to assess the tool's error.

– Power analysis after place and route with an activity file covering the combinational logic, to raise confidence to a level usable for formal comparison.

– Porting to a smaller device of the same generation: as the core uses only 5% of the logic, most static power is a cost of the device, not the design.

**Extending the capability of the model:**

– Supporting more leads and rhythm groups, using the resource headroom to replicate the parallel units while keeping the weights on chip.

– Classifying by sliding window to localize an abnormality, rather than one label for the whole record.

– Recalibrating the decision thresholds on the target dataset to improve far transfer — Section 4.2.2 shows the loss comes chiefly from a threshold shift, not from poorer separability.

**Upgrading the hardware architecture:**

– Pipelining the layers so a later layer starts as soon as the previous one has enough data, replacing the present sequential run, to cut end-to-end latency.

– Studying different degrees of parallelism to obtain the area–latency–energy trade-off curve and select an operating point per deployment constraint.

**Towards a complete system:**

– Coupling the hardware to an electrocardiogram acquisition circuit so that the system operates on a continuous real-time data stream.

– Adding preprocessing in hardware — noise filtering and baseline wander removal — so the circuit can work on directly acquired signals.

## REFERENCES


[1]. J. Zheng, J. Zhang, S. Danioko, H. Yao, H. Guo, and C. Rakovski, "A 12-lead electrocardiogram database for arrhythmia research covering more than 10,000 patients," Scientific Data, vol. 7, art. 48, 2020. DOI: 10.1038/s41597-020-0386-x.

[2]. E. A. Perez Alday, A. Gu, A. J. Shah, C. Robichaux, A.-K. I. Wong, C. Liu, F. Liu, A. B. Rad, A. Elola, S. Seyedi, Q. Li, A. Sharma, G. D. Clifford, and M. A. Reyna, "Classification of 12-lead ECGs: the PhysioNet/Computing in Cardiology Challenge 2020," Physiological Measurement, vol. 41, no. 12, art. 124003, 2020. DOI: 10.1088/1361-6579/abc960.

[3]. S. Kiranyaz, T. Ince, and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," IEEE Transactions on Biomedical Engineering, vol. 63, no. 3, pp. 664–675, 2016. DOI: 10.1109/TBME.2015.2468589.

[4]. A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, et al., "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," Nature Medicine, vol. 25, no. 1, pp. 65–69, 2019. DOI: 10.1038/s41591-018-0268-3.

[5]. K.-H. Le, H.-H. Pham, T.-B. Nguyen, et al., "LightX3ECG: A lightweight and explainable deep learning system for 3-lead electrocardiogram classification," Biomedical Signal Processing and Control, vol. 85, art. 104963, 2023. DOI: 10.1016/j.bspc.2023.104963.

[6]. T. Yoon and D. Kang, "Bimodal CNN for cardiovascular disease classification by co-training ECG grayscale images and scalograms," Scientific Reports, vol. 13, art. 2937, 2023. DOI: 10.1038/s41598-023-30208-8.

[7]. D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, 2015. arXiv:1412.6980.

[8]. M. Lin, Q. Chen, and S. Yan, "Network in network," in Proc. 2nd Int. Conf. Learning Representations (ICLR), 2014. arXiv:1312.4400.

[9]. S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 28, pp. 1135–1143, 2015. DOI: 10.5555/2969239.2969366.

[10]. P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," in Proc. 5th Int. Conf. Learning Representations (ICLR), 2017. arXiv:1611.06440.

[11]. H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in Proc. 5th Int. Conf. Learning Representations (ICLR), 2017. arXiv:1608.08710.

[12]. B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713, 2018. DOI: 10.1109/CVPR.2018.00096.

[13]. Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

[14]. D. Miyashita, E. H. Lee, and B. Murmann, "Convolutional neural networks using logarithmic data representation," arXiv:1603.01025, 2016.

[15]. M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," arXiv:2106.08295, 2021.

[16]. Y. Liu, et al., "A fully-mapped and energy-efficient FPGA accelerator for dual-function AI-based analysis of ECG," Frontiers in Physiology, vol. 14, art. 1079503, 2023. DOI: 10.3389/fphys.2023.1079503.

[17]. H. L. Pham, T. D. Tran, V. T. D. Le, and Y. Nakashima, "MINA: A hardware-efficient and flexible Mini-InceptionNet accelerator for ECG classification in

wearable devices," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 6, pp. 2740–2753, 2025. DOI: 10.1109/TCSI.2025.3553837.

[18]. V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017. DOI: 10.1109/JPROC.2017.2761740.

[19]. C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA), pp. 161–170, 2015. DOI: 10.1145/2684746.2689060.

[20]. S. I. Venieris, A. Kouris, and C.-S. Bouganis, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," ACM Computing Surveys, vol. 51, no. 3, art. 56, pp. 1–39, 2018. DOI: 10.1145/3186332.

[21]. S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Prentice Hall PTR, 2003. ISBN: 0-13-044911-3.

[22]. Intel Corporation, Cyclone V Device Handbook, Volume 1: Device Interfaces and Integration, Intel/Altera, 2020.

[23]. A. Tran, K. Tran, and C. Do, "ECG-LDC: A hardware-efficient low-dimensional computing framework for ECG arrhythmia classification," *arXiv preprint* arXiv:2607.09680, 2026.